

StreamKM++ sur Apache Spark

Portage d'une implémentation Flink vers Spark

Groupe Jules MAURICE, Asma BEN-ZINE, Victorien GBAGUIDI

Août 2026

Table des matières

1	Introduction	3
2	Algorithmes	5
2.1	Algorithme de Lloyd pondéré	5
2.2	Seeding <i>k</i> -means++	6
2.3	Construction du coreset : le coreset tree	6
2.4	Merge-and-reduce	7
3	Apache Spark	9
3.1	RDD et DataFrame	9
3.2	Micro-batch et vrai streaming	9
3.3	Diffusion (broadcast)	10
3.4	treeReduce et treeAggregate	10
3.5	Partitions et cœurs : deux leviers découplés	10
4	Implémentation	11
4.1	StreamKM++ distribué, flux borné	11
4.1.1	Coresets partiels par partition	11
4.1.2	Fusion arborescente du coreset final	11
4.1.3	Boucle Structured Streaming complète	12
4.1.4	Table de correspondance	12
4.2	Méthodologie d'évaluation du coût	12
4.3	Variante à requêtes périodiques (décrite, non implémentée)	13
4.4	Ce qui se traduit depuis Flink, ce qui ne se traduit pas	14
5	Évaluation expérimentale	15
5.1	Protocole et environnement	15
5.1.1	Optimisation du kernel local (JMH)	15
6	Discussion — points forts et points faibles	17
6.1	Points forts	17
6.2	Points faibles	17
7	Conclusion et perspectives	19
7.1	Travaux futurs	19
.1	Code source complet	20
.1.1	Configuration du projet	20
.1.2	Couche core : structures de données et k-means	21
.1.3	Couche coreset : coreset tree et merge-and-reduce	26
.1.4	Couche spark : portage distribué	34
.1.5	Couche experiments : harnais de mesure de qualité	37
.1.6	Fonction Main	40

.1.7	Suite de tests	42
------	--------------------------	----

Chapitre 1

Introduction

Le clustering k -means classique suppose que l'ensemble des points tient en mémoire et peut être relu autant de fois que nécessaire. Cette hypothèse s'effondre dès que les données arrivent en flux continu : un capteur, un flux de clics, un flux de transactions. On ne peut ni tout stocker, ni relire deux fois le même point, et pourtant on veut un clustering de qualité comparable à ce qu'on obtiendrait en traitant les données par lots.

STREAMKM++ (Ackermann et al., 2012) répond à ce problème par la technique du *coreset* : au lieu de conserver le flux, on en maintient un résumé pondéré de taille bornée m , construit incrémentalement, tel que le coût de clustering calculé sur ce résumé approxime le coût de clustering sur les données complètes, pour n'importe quel jeu de centres (Définition 2.2 de la thèse de référence). Un seul passage sur les données suffit, et la mémoire utilisée ne dépend que de m et $\log(n)$, jamais de n directement.

Objet de ce rapport. Bitsakis (2018) implémente StreamKM++ dans Apache Flink et l'évalue de façon approfondie (chapitres 4 et 5 de sa thèse). Notre travail consiste à **porter cette implémentation vers Apache Spark**, dans le cadre du cours de déploiement ML à grande échelle de ce M2. Ce portage n'est pas une simple traduction syntaxique : Flink et Spark reposent sur des modèles d'exécution différents (état par *subtask* adressable vs. RDD immuables et micro-batch), et une partie du travail intéressant de ce rapport consiste précisément à identifier ce qui se traduit directement, ce qui demande une réinterprétation, et ce qui n'a tout simplement pas d'équivalent côté Spark (chapitre 4, §4.4).

Méthode. Nous avons suivi la même progression que la thèse de référence, dans le même ordre : comprendre la théorie avant d'écrire du code (chapitre 2), décrire le modèle d'exécution cible avant de l'utiliser (chapitre 3), implémenter opérateur par opérateur en partant d'une version simple avant d'ajouter des raffinements (chapitre 4), et enfin valider la *qualité* du clustering avant de mesurer sa *scalabilité* (chapitre 5) — mesurer un speedup sur un algorithme dont le résultat serait faux n'aurait aucun sens. Cet ordre n'est pas seulement une méthode de travail : c'est aussi le plan de ce rapport.

Architecture logicielle. Le code est organisé en trois couches strictement séparées, une contrainte de conception que nous avons tenue tout au long du projet :

- `streamkm.core` — k -means, k -means++, distance euclidienne au carré. Pur Scala, aucune dépendance.
- `streamkm.coreset` — l'arbre de coreset et la technique merge-and-reduce. Pur Scala, sérialisable, testable sans JVM Spark.
- `streamkm.spark` — l'orchestration Spark (RDD, `treeReduce`, Structured Streaming). Aucune logique algorithmique n'y vit : cette couche ne fait que distribuer des appels aux deux précédentes.

Cette séparation permet de tester et de faire du micro-benchmarking sur les deux premières couches indépendamment de tout cluster Spark (chapitre 5, §??), et elle rend visible, dans le code même, la frontière entre « ce qui est StreamKM++ » et « ce qui est Spark ».

Chapitre 2

Algorithmes

StreamKM++ répond à un problème que le k -means classique ne peut pas traiter tel quel : un flux de points trop volumineux pour tenir en mémoire, qu'on ne peut lire qu'une seule fois. La solution retenue par Ackermann et al. (2012) et reprise par Bitsakis (2018) consiste à maintenir en permanence un *coreset* — un résumé pondéré de taille bornée m , construit incrémentalement, tel que le clustering optimal du coreset approxime le clustering optimal des données complètes (Définition 2.2, thèse). Ce chapitre décrit les quatre briques qui rendent cela possible, et pour chacune, les décisions prises par le groupe lorsque la thèse laissait une ambiguïté. Le code correspondant vit dans `streamkm.core` (§2.1–2.2) et `streamkm.coreset` (§2.3–2.4) ; le portage de ces briques vers Spark fait l'objet du chapitre 4.

2.1 Algorithme de Lloyd pondéré

L'algorithme de Lloyd (thèse, algorithme 2.1) alterne deux étapes jusqu'à convergence : affecter chaque point au centre le plus proche, puis recentrer chaque centre sur la moyenne de son cluster.

Algorithm 1 Lloyd pondéré (thèse, algo. 2.1 — `KMeans.lloyd`)

Require: points pondérés $P = \{(p_i, w_i)\}_{i=1}^n$, centres initiaux $C = \{c_1, \dots, c_k\}$

1: **repeat**

2: **for** chaque point $p_i \in P$ **do**

3: affecter p_i au centre c_j le plus proche (distance au carré)

4: **end for**

5: **for** chaque cluster j **do**

6: $c_j \leftarrow \frac{\sum_{p_i \in \text{cluster } j} w_i \cdot p_i}{\sum_{p_i \in \text{cluster } j} w_i}$

7: **end for**

8: **until** $|\Delta \text{coût}| < \text{tol}$ ou maxIter atteint

Ensure: centres C , coût = $\sum_i w_i \cdot \min_j \|p_i - c_j\|^2$

Pourquoi « pondéré » n'est pas un détail. La thèse énonce l'algorithme sur des points bruts, tous de poids 1. Mais dans StreamKM++, Lloyd n'est *jamaïs* appliqué qu'à un coreset — un ensemble où le poids d'un point représente la masse d'un sous-ensemble entier de points originaux (cf. §2.3). Un centre recalculé comme une moyenne *non* pondérée produirait un résultat silencieusement faux : le code compile, l'algorithme converge, mais vers un optimum qui n'a plus de rapport avec les données réelles. C'est le piège classique de StreamKM++, identifié dès la phase de conception du groupe (METHODODO, étape E1) et vérifié par un test dédié (`KMeansSpec`, « un point de poids w équivaut à w copies du même point ») : un point de poids 3 doit produire exactement le même centre que trois copies de poids 1 du même point.

Un invariant vérifié en continu. Le coût de Lloyd décroît de façon monotone à chaque itération — propriété classique de l’algorithme, mais aussi le signal de correction le plus direct en cas de bug d’affectation ou de pondération. `KMeans.lloyd` le vérifie par une assertion active en mode debug (`-ea`), plutôt que de faire confiance silencieusement à la convergence numérique.

Clusters vides. Si un cluster ne reçoit aucun point à une itération, son centre est conservé tel quel plutôt que ré-initialisé. Ce choix, plus simple que l’alternative classique (ré-amorcer sur le point le plus éloigné), a aussi l’avantage de préserver la monotonie du coût — un cluster vide contribue 0, une ré-initialisation aurait pu faire remonter le coût et casser l’invariant ci-dessus.

2.2 Seeding k -means++

Une initialisation aléatoire de Lloyd converge souvent vers un optimum local médiocre. Le seeding k -means++ (thèse, algorithme 2.2) choisit les centres initiaux avec une probabilité proportionnelle au carré de leur distance au centre déjà choisi le plus proche (échantillonnage D^2), ce qui répartit les centres sur les amas plutôt que de les laisser s’accumuler.

Algorithm 2 Seeding D^2 pondéré (thèse, algo. 2.2 lignes 1–3 — `KMeans.seedPlusPlus`)

Require: points pondérés P , nombre de centres k

- 1: $c_1 \leftarrow$ un point de P tiré *proportionnellement à son poids* ▷ voir remarque ci-dessous
- 2: **for** $c \leftarrow 2$ à k **do**
- 3: $closest(p) \leftarrow \min_{j < c} \|p - c_j\|^2$ pour chaque $p \in P$
- 4: $c_c \leftarrow$ un point $p \in P$ tiré avec probabilité $\propto w_p \cdot closest(p)$
- 5: **end for**

Ensure: k centres $\{c_1, \dots, c_k\}$

Écart assumé par rapport à la thèse : premier centre non uniforme. L’algorithme 2.2 (ligne 1) tire le tout premier centre *uniformément* parmi les points. Notre implémentation le tire proportionnellement au poids. Sur des points bruts (tous de poids 1) les deux tirages coïncident exactement — l’écart n’a donc *aucun effet* sur l’algorithme 2.2 pris isolément. Mais dans le pipeline complet de StreamKM++, k -means++ n’est jamais appliqué qu’à un coreset (thèse, §2.3.4, étape 2), où les poids sont très hétérogènes : un tirage uniforme y biaiserait systématiquement l’initialisation vers les régions les moins densément représentées par les données d’origine. Le tirage proportionnel au poids est donc la généralisation correcte de la ligne 1 au cas pondéré, pas une simplification. La même remarque s’applique, pour la même raison, au premier représentant du coreset tree (§2.3).

Complexité. Le calcul incrémental de $closest(p)$ — mis à jour à chaque nouveau centre plutôt que recalculé depuis zéro — donne un coût en $O(n \cdot k \cdot d)$ et non $O(n \cdot k^2 \cdot d)$.

Combiner les deux : `KMeans.fit`. La thèse (§2.3.4, étape 2) applique cinq fois la combinaison seeding + Lloyd et retient le clustering de coût minimal — c’est `KMeans.fit(points, k, nRestarts=5)`. Le §5.3.1 de la thèse montre qu’une seule application suffit en pratique ; `nRestarts` reste un paramètre expérimental (E3, ch. 5), pas une constante algorithmique figée.

2.3 Construction du coreset : le coreset tree

Un coreset au sens de la Définition 2.2 (thèse) peut être obtenu par un tirage D^2 direct sur l’ensemble des points (`AdaptiveCoreset`, algo. 2.3), mais au prix d’une complexité $\Theta(dnm)$ — chaque nouveau point du coreset impose de parcourir tous les points d’origine. Le *coreset tree*

(thèse, §2.3.3, algo. 2.4) ramène cette complexité à $O(dm^2)$ en organisant la construction comme un clustering hiérarchique divisif : chaque nœud de l'arbre porte un sous-ensemble de points et un représentant, la racine porte l'ensemble complet, et chaque scission raffine localement le coreset sans retoucher les autres branches.

Algorithm 3 Coreset tree (thèse, algo. 2.4 — `CoresetTree.build`)

Require: points pondérés P , taille cible $m \leq |P|$

- 1: $q_1 \leftarrow$ un point de P tiré proportionnellement à son poids ▷ cf. remarque §2.2
- 2: racine $\leftarrow$ nœud unique contenant P , représentant q_1
- 3: **for** $i \leftarrow 2$ à m **do**
- 4: $\ell \leftarrow$ descendre de la racine à une feuille, un fils choisi $\propto$ son coût ▷ §2.3
- 5: $q_i \leftarrow$ un point de la feuille ℓ tiré $\propto$ sa distance au carré au représentant de ℓ (éch. D^2)
- 6: scinder ℓ en deux nouvelles feuilles (une par représentant q_ℓ, q_i)
- 7: remonter la mise à jour du coût/de la masse jusqu'à la racine
- 8: **end for**

Ensure: $S = \{q_1, \dots, q_m\}$, chaque q_i pondéré par la masse de sa feuille

L'ambiguïté centrale de la thèse, et comment on l'a tranchée. La thèse attache à chaque nœud v un attribut $\text{weight}(v)$, défini juste avant l'algorithme 2.4 comme $\text{cost}(P_v, q_v)$ — une somme de distances au carré — mais réutilise ensuite ce *même mot* « weight », au §2.3.4, pour désigner la masse (le nombre de points) que représente chaque point du coreset final. Ce sont deux quantités de nature complètement différente portant le même nom. Notre implémentation les sépare explicitement dans `CoresetTree.Node` : le champ `cost` (la somme de distances au carré, qui pilote la descente dans l'arbre, étape 1 du §2.3.3) et le champ `mass` (la masse, qui devient le poids du point de sortie). Les confondre produit un algorithme qui compile, tourne sans erreur, et construit un coreset dont les poids sont faux — un bug purement silencieux.

Une seconde ambiguïté découle directement de la première : le §2.3.3, étape 1 dit en prose que la descente choisit une feuille « avec une probabilité proportionnelle à $\text{cost}(P_\ell, q_\ell)$ », mais la ligne 5 de l'algorithme 2.4 dit « selon leurs *weights* ». En comprenant $\text{weight}(v)$ au sens de sa définition (= `cost`), les deux formulations concordent — c'est cette lecture que confirme le papier original d'Ackermann et al., et c'est donc le **coût**, pas la masse, qui pilote la descente dans `CoresetTree.descend`. Le raisonnement le confirme aussi a posteriori : une feuille déjà bien approximée par son représentant (coût faible) n'a pas besoin d'être raffinée davantage ; utiliser la masse à la place ferait dériver l'algorithme vers un simple échantillonnage proportionnel à la densité, ce qui lui ferait perdre sa propriété de coreset.

Invariant vérifié. La somme des poids en sortie du coreset tree est *exactement* égale à la somme des poids en entrée — c'est le test le plus discriminant de toute la couche **coreset** (`CoresetSpec`, « conserve exactement la masse totale ») : un coreset dont la masse ne se conserve pas est faux, quelle que soit par ailleurs la plausibilité de ses centres.

2.4 Merge-and-reduce

Le coreset tree construit un coreset à partir d'un ensemble de points connu d'avance. Pour un *flux*, StreamKM++ ajoute une seconde couche : la technique merge-and-reduce (thèse, §2.3.4, algo. 2.5), qui maintient un ensemble de « seaux » (*buckets*) $B_0, B_1, \dots$ et applique le coreset tree pour les fusionner à la demande, sans jamais avoir besoin de connaître n à l'avance.

Algorithm 4 Insertion dans le flux (thèse, algo. 2.5 — `BucketManager.insert`)

Require: point p , taille de coreset m

```
1: insérer  $p$  dans  $B_0$ 
2: if  $B_0$  est plein ( $m$  points) then
3:    $S \leftarrow$  ensemble vide,  $i \leftarrow 1$ 
4:   déplacer les points de  $B_0$  dans  $S$ , vider  $B_0$ 
5:   while  $B_i$  est non vide do
6:      $S \leftarrow \text{CORESETTREE}(S \cup B_i, m)$  ▷ réduction à  $m$  points
7:     vider  $B_i$ ;  $i \leftarrow i + 1$ 
8:   end while
9:   déplacer  $S$  dans  $B_i$ 
10: end if
```

Invariants (thèse, §2.3.4, propriétés 1–2), vérifiés par `checkInvariants`. B_0 contient entre 0 et m points bruts (poids 1); pour $i \geq 1$, B_i est soit vide, soit contient *exactement* m points, et représente alors $2^{i-1} \cdot m$ points du flux d’origine. Le nombre de buckets alloués croît en $O(\log(n/m))$: les niveaux sont créés à la demande, sans jamais avoir besoin de n a priori — condition essentielle pour un flux dont la taille finale est inconnue.

Le coreset final. À tout instant, l’union de tous les buckets non vides (`BucketManager.union`) constitue un résumé valide de tout ce qui a été vu jusqu’ici, borné par $m \cdot (\lceil \log_2(n/m) \rceil + 2)$ points pondérés (thèse, §2.3.4, clarification 2). Interroger cette union n’interrompt pas la consommation du flux : c’est ce qui permettra, en extension future (ch. 4, §4.3), de ré-évaluer le clustering périodiquement sans tout reconstruire.

Fusion de deux structures indépendantes. La thèse observe (§2.3.4) que (1) l’union de deux ε -coresets d’ensembles disjoints est un ε -coreset de leur union, et que (2) un ε -coreset d’un δ -coreset est un $((1 + \varepsilon)(1 + \delta) - 1)$ -coreset — l’erreur se compose *multiplicativement*, pas additivement, à chaque niveau de fusion. `BucketManager.merge` implémente exactement cette opération pour deux managers indépendants (par exemple, deux partitions distinctes d’un même flux), sans modifier ni l’un ni l’autre. Le résultat est un manager « *collapsé* » — un coreset unique de taille m , plus un simple empilement de buckets valides — dont toute insertion ultérieure est délibérément refusée : sémantiquement, un manager fusionné est un résultat, pas un flux qu’on continuerait à alimenter. Cette opération est le point de départ direct du portage vers Spark (ch. 4) : elle a exactement la signature d’un `combOp` de `treeReduce`, ce qui explique pourquoi la couche `streamkm.spark` n’a eu besoin d’aucune modification de `streamkm.coreset` pour distribuer merge-and-reduce sur un cluster.

Chapitre 3

Apache Spark

La thèse de référence consacre son chapitre 3 à décrire Flink *avant* d'en présenter l'implémentation (chapitre 4), parce que le dataflow proposé est directement contraint par ce que le modèle d'exécution sait faire : état par *subtask*, *watermarks*, diffusion (*broadcast*). Nous suivons la même logique avec Spark : ce chapitre décrit les mécanismes sur lesquels s'appuie notre implémentation (chapitre 4), avant de la présenter.

3.1 RDD et DataFrame

Spark expose deux abstractions principales pour les données distribuées. Un **RDD** (*Resilient Distributed Dataset*) est une collection immuable, partitionnée, d'objets Scala arbitraires — c'est l'abstraction que nous utilisons pour manipuler des `streamkm.core.Point`, une classe qui n'a pas de schéma de colonnes et ne bénéficie donc d'aucune optimisation particulière à être représentée en **DataFrame**. Un **DataFrame** est une table distribuée typée par un schéma de colonnes, optimisée par le moteur Catalyst (plan logique, plan physique, génération de code) — c'est l'abstraction que nous utilisons pour la source du flux (`streamkm.spark.StreamKMPlusPlus.run`), avant de repasser en RDD dès que le traitement touche à la logique de STREAMKM++ elle-même.

Ce choix — **DataFrame** en entrée, RDD dès qu'on entre dans notre propre logique — est délibéré : `Point` n'est pas une `case class` (cf. `streamkm.core.Point`), et lui construire un **Encoder** Spark pour bénéficier de Catalyst sur ce type n'aurait apporté aucun gain (aucune requête, aucune agrégation column-oriented n'est faite sur un `Point`), au prix d'un couplage supplémentaire entre `core` et Spark — exactement ce que la « règle d'or » de notre architecture (§1) interdit.

3.2 Micro-batch et vrai streaming

Flink est un moteur de *vrai streaming* : chaque événement est traité dès son arrivée, avec un état persistant entre événements. Spark Structured Streaming, malgré son nom, reste fondamentalement un moteur de *micro-batch* : le flux est découpé en petits lots (déclenchés par un **Trigger**, par exemple un intervalle de temps ou l'arrivée d'un fichier), et chaque lot est traité comme un **DataFrame** borné ordinaire, via `foreachBatch`. Cette différence de modèle a une conséquence directe et positive pour notre portage : la thèse doit détecter elle-même la fin d'un fichier en interceptant le mécanisme de *watermark* de Flink (opérateur `HandleWatermark`, §4.1 de la thèse) ; avec Spark, la frontière entre deux lots est **explicite** — `foreachBatch` est appelé une fois par micro-batch, sans bricolage nécessaire. C'est le premier exemple concret, développé au chapitre 4, de ce qui se simplifie dans le portage.

En contrepartie, le micro-batch introduit une latence structurelle qu'un vrai moteur de streaming n'a pas : un point n'est traité qu'à la clôture de son lot, jamais immédiatement à son arrivée. Nous revenons sur ce compromis au chapitre 6.

3.3 Diffusion (broadcast)

Quand une valeur doit être lue, identique, par toutes les tâches d’un job (par exemple un jeu de centres déjà calculé), l’envoyer une fois par tâche gaspillerait de la bande passante réseau proportionnellement au nombre de partitions. `SparkContext.broadcast` envoie la valeur une seule fois par exécutateur (et non par tâche), et la met en cache localement pour toute la durée de vie du `Broadcast`. Nous l’utilisons dans `streamkm.spark.CostEvaluator.cost` pour diffuser les centres avant de calculer, en parallèle, le coût de chaque partition (chapitre 4, §4.2) — l’équivalent Spark de l’opérateur `FlatMapToCentroid + .broadcast()` de la thèse (§4.2).

3.4 `treeReduce` et `treeAggregate`

Réduire les résultats de N partitions vers le *driver* peut se faire de deux façons : **linéairement** (chaque partition envoie son résultat directement au driver, qui les combine un par un — $O(N)$ étapes séquentielles sur le driver), ou en **arbre** (`treeReduce`/`treeAggregate` : les résultats sont combinés par paires à travers plusieurs paliers intermédiaires, $O(\log N)$ étapes séquentielles, le reste en parallèle). Pour un grand nombre de partitions, la réduction linéaire concentre tout le travail de fusion sur un seul nœud — exactement le défaut que la thèse identifie dans son propre opérateur `FlatMapToFinalCoreset` (§4.1, non-parallèle par construction) comme le goulot qui plafonne le speedup au-delà de 8 cœurs (§5.3.1). C’est le point central de notre apport par rapport à la thèse (chapitre 4, §4.1) : nous remplaçons cette fusion séquentielle par une fusion en arbre.

Nous utilisons `treeReduce` plutôt que `treeAggregate` pour la fusion des coresets : `treeAggregate(zeroValue, combOp, depth)` clone un `zeroValue` *unique* (donc une seule graine aléatoire) pour chaque partition, ce qui casserait la reproductibilité par partition requise par notre protocole expérimental (chapitre 5, §??). `mapPartitionsWithIndex` (une graine dérivée de l’indice de partition) suivi de `treeReduce` réalise la même opération — un accumulateur construit une fois par partition, fusionné en arbre — sans ce biais. Ce choix, documenté dans le code (`StreamKMPlusPlus.mergeCoresets`), est un écart assumé par rapport à l’exemple de pipeline de notre spécification interne, pas par rapport à ses invariants.

3.5 Partitions et cœurs : deux leviers découplés

Chez Flink, le *parallélisme* est un seul nombre : il fixe à la fois le nombre de *subtasks* et le nombre de listes de *buckets* merge-and-reduce actives. Chez Spark, ces deux choses sont **découplées** : le **nombre de partitions** détermine le nombre d’accumulateurs `BucketManager` indépendants (donc la *qualité* — plus de partitions, plus de coresets partiels, un raffinement plus fin) ; le **nombre de cœurs** détermine le *débit* de traitement (donc le *temps*). La thèse observe que le coût *diminue* quand le parallélisme Flink augmente (§5.3.1, fig. 5.9) et l’attribue à la multiplication des listes de buckets — mais elle ne peut pas isoler cet effet de celui du nombre de cœurs, puisque les deux varient ensemble chez Flink. Le découplage de Spark permet de tester cette explication directement, en fixant les cœurs et en faisant varier uniquement les partitions (chapitre 5, §??, expérience E3bis) — une vérification que le modèle de Flink ne permet tout simplement pas de faire, et qui constitue l’une de nos contributions propres à ce rapport. En pratique, cela signifie que `spark.sql.shuffle.partitions` et le nombre de partitions des RDD manipulés sont toujours fixés explicitement dans notre code (`repartition(N)`), jamais laissés à la décision de Spark.

Chapitre 4

Implémentation

Ce chapitre décrit la couche `streamkm.spark`, opérateur par opérateur, en suivant le même plan que le chapitre 4 de la thèse : d’abord le dataflow borné (§4.1, miroir du §4.1 de la thèse), puis la méthodologie d’évaluation du coût (§4.2, miroir du §4.2), puis la variante à requêtes périodiques (§4.3, miroir du §4.3, *décrite* plutôt qu’implémentée — cf. justification en fin de section), et enfin un bilan de ce qui se traduit ou non depuis Flink (§4.4). Rappel de la règle d’or (chapitre 1) : aucun des fichiers décrits ici ne contient de logique algorithmique propre — tout le calcul vient de `streamkm.core` et `streamkm.coreset` (chapitre 2), déjà testés indépendamment de Spark.

4.1 StreamKM++ distribué, flux borné

4.1.1 Coresets partiels par partition

`StreamKMPlusPlus.partialCoresets` porte l’opérateur `FlatMapToPartialCoresets` de la thèse (§4.1) : chaque subtask Flink « consomme les points d’entrée selon l’algorithme 2.5 » — une partition Spark joue exactement ce rôle.

```
def partialCoresets(rdd: RDD[Point], p: StreamKMParams): RDD[Array[Point]] =  
  rdd.mapPartitionsWithIndex { (idx, it) =>  
    val bm = new BucketManager(p.m, p.seed + idx)  
    bm.insertAll(it)  
    Iterator.single(bm.extractCoreset())  
  }
```

La graine de chaque `BucketManager` est dérivée de l’indice de partition (`p.seed + idx`) plutôt que partagée entre partitions — condition nécessaire pour que l’expérience E3bis (chapitre 5) reste interprétable : avec une graine partagée, changer le nombre de partitions changerait aussi la suite aléatoire consommée par chaque partition, confondant l’effet mesuré (le partitionnement) avec un simple changement de graine.

4.1.2 Fusion arborescente du coreset final

`StreamKMPlusPlus.mergeCoresets` remplace l’opérateur `FlatMapToFinalCoreset` de la thèse — l’opérateur **non parallèle** par construction (§4.1 : fusion séquentielle, un coreset partiel à la fois) que la thèse elle-même identifie comme le goulot qui plafonne le speedup au-delà de 8 cœurs (§5.3.1). C’est notre apport principal par rapport à la thèse, rendu possible par le découplage partitions/cœurs de Spark (§3.4) :

```
def mergeCoresets(rdd: RDD[Point], p: StreamKMParams, depth: Int = 2): Array[Point] = {
```

```

val perPartition: RDD[BucketManager] = rdd.mapPartitionsWithIndex { (idx, it) =>
  val bm = new BucketManager(p.m, p.seed + idx)
  bm.insertAll(it)
  Iterator.single(bm)
}
perPartition.treeReduce((a, b) => a.merge(b), depth).extractCoreset()
}

```

Le `combOp` de ce `treeReduce` est `BucketManager.merge` (chapitre 2, §2.4), réutilisé *tel quel* — aucune extension de `streamkm.coreset` n’a été nécessaire pour distribuer merge-and-reduce sur un cluster, ce qui confirme a posteriori la séparation stricte des couches annoncée en introduction.

4.1.3 Boucle Structured Streaming complète

`StreamKMPlusPlus.run` assemble l’ensemble du dataflow — de l’équivalent de `HdfsSource` jusqu’à l’équivalent de `FlatMapToKmeansPP` — autour d’un état global unique (`BucketManager`) maintenu côté *driver* entre les micro-batches :

```

def run(source: DataFrame, p: StreamKMParams,
        onModel: KMeansModel => Unit,
        featuresCol: String = "features"): StreamingQuery = {

  val global = new BucketManager(p.m, p.seed)

  source.writeStream.foreachBatch { (batch, batchId) =>
    val points = batch.rdd.map(row => Point(row.getAs[Seq[Double]](featuresCol).toArray))
    val partial = mergeCoresets(points, p.copy(seed = p.seed + batchId))
    partial.foreach(global.insert)
    val model = KMeans.fit(global.extractCoreset(), p.k, p.nRestarts, seed = p.seed)
    onModel(model)
  }.start()
}

```

Chaque micro-batch produit son propre coreset distribué (`mergeCoresets`), qui est ensuite intégré dans l’état global par `insert` — exactement la même opération de fusion que `FlatMapToFinalCoreset`, appliquée cette fois entre micro-batches plutôt qu’entre partitions. Le contrat d’entrée (une colonne `featuresCol` de type `array<double>`, poids implicite 1.0) reflète directement `Point.scala` (chapitre 2) : le poids n’existe que dans les coresets produits en interne, jamais dans les données brutes du flux.

4.1.4 Table de correspondance

4.2 Méthodologie d’évaluation du coût

La thèse (§4.2) recalcule le coût d’un clustering en parallèle, à partir d’un fichier de centres déjà connu, dans un job Flink séparé — nécessaire pour mesurer la qualité sur de gros volumes sans que ce calcul devienne lui-même le goulot d’étranglement des expériences du chapitre 5. Elle utilise pour cela un *connected stream* (deux sources fusionnées : les points et les centres), un opérateur de diffusion et un opérateur de somme partielle. Cette machinerie tient, côté Spark, en trois lignes :

```

def cost(points: RDD[Point], centers: Array[Array[Double]], depth: Int = 2): Double = {
  val bcCenters = points.sparkContext.broadcast(centers)

```

Opérateur (thèse)	Flink	Rôle	Équivalent Spark
HdfsSource		lecture parallèle des lignes	source fichier partitionnée
HandleWatermark		détecter la fin de fichier	aucun — <code>foreachBatch</code> rend la frontière explicite
FlatMapToPoint		parsing <code>String</code> $\rightarrow$ <code>Point</code>	<code>mapPartitions</code> + <code>filter</code>
FlatMapToPartialCoresets		merge-and-reduce par subtask	<code>partialCoresets</code>
FlatMapToFinalCoreset		fusion séquentielle (par. 1)	<code>mergeCoresets</code> (<code>treeReduce</code>)
FlatMapToKmeansPP		$5 \times k$ -means++ (par. 1)	<code>KMeans.fit</code> sur le driver
HdfsSink		écriture des centres	sink (fichier ou <code>onModel</code>)

TABLE 4.1 – Correspondance des opérateurs, dataflow borné (§4.1 de la thèse).

```

points
  .mapPartitions(it => Iterator.single(KMeans.cost(it.toArray, bcCenters.value)))
  .treeReduce(_ + _, depth)
}

```

`sc.broadcast(centers)` remplace `FlatMapToCentroid` + `.broadcast()`; `mapPartitions(... KMeans.cost ...)` remplace `CoFlatMapToPartialCost`; `treeReduce(_+_)` remplace `CoFlatMapToFinalCost`. Contrairement à la fusion de coresets (§4.1), cette agrégation est une simple somme — associative et exacte à la précision flottante près — ce qui explique pourquoi elle ne nécessite aucune des précautions de reproductibilité par partition discutées plus haut : le résultat ne dépend pas du nombre de partitions choisi. C’est cet instrument qui sert de mesure à **toutes** les expériences du chapitre 5 — jamais le coût mesuré sur le coreset lui-même, qui n’est pas comparable au coût sur les données complètes.

4.3 Variante à requêtes périodiques (décrite, non implémentée)

Le §4.3 de la thèse propose une variante où le clustering est ré-évalué périodiquement plutôt qu’une seule fois à la fin du flux, et où plusieurs valeurs de k sont calculées simultanément. Faute de temps avant la deadline, cette variante **n’a pas été implémentée** — conformément à notre méthodologie de travail, qui préférerait explicitement documenter cette limite plutôt que de livrer une implémentation bâclée. La thèse elle-même ne l’évalue pas expérimentalement (elle le signale comme travail futur, ch. 6), ce qui rend ce choix d’autant plus défendable.

Le portage resterait néanmoins direct avec ce que nous avons déjà construit : le *trigger* de Structured Streaming remplacerait directement l’opérateur `SourceRequests_R` de génération de requêtes périodiques (`Trigger.ProcessingTime("30 seconds")` au lieu d’un flux de requêtes explicite); `BucketManager.extractCoreset` — qui ne vide jamais l’état, une propriété déjà exploitée par `run` — permettrait de dupliquer le coreset courant pour plusieurs valeurs de k simultanément (`Seq(25, 50, 75).map(k => KMeans.fit(coreset, k))`), remplaçant `FlatMapToFinalCoreset_R`. Seule la persistance du meilleur clustering *par* k (`ValueState` chez Flink) demanderait une structure nouvelle côté Spark (une simple `Map[Int, KMeansModel]` tenue par le `onModel` de `run` suffirait).

Le §4.3.1 de la thèse (*Queryable State*, interrogation en temps réel de l’état interne depuis l’extérieur du job) **n’a en revanche pas d’équivalent Spark** : Spark n’expose aucun état inter-micro-batch interrogeable depuis l’extérieur. C’est un avantage architectural réel de Flink, que nous assumons comme tel — voir la discussion du chapitre 6. Le contournement usuel (un sink `memory` interrogé par `spark.sql("SELECT * FROM centers")`, ou l’écriture des centres dans un store externe) n’offre pas la même garantie de fraîcheur immédiate.

4.4 Ce qui se traduit depuis Flink, ce qui ne se traduit pas

Ce paragraphe de synthèse répond directement à la question posée par l'énoncé — porter en Spark une solution Flink — et n'a pas d'équivalent dans la thèse : c'est notre contribution propre.

- **Disparaît entièrement** : `HandleWatermark`. Le bricolage watermark/marqueur « EOF » de la thèse n'a plus de raison d'être — `foreachBatch` rend la frontière de micro-batch explicite par construction (§3).
- **Se traduit avec un gain net** : la fusion des coreset partiels. L'opérateur non-parallèle de la thèse (§4.1) devient une réduction en arbre (`treeReduce`), ce qui repousse directement le plafond de speedup identifié au §5.3.1 de la thèse — à confirmer expérimentalement (chapitre 5, expérience E7).
- **Se traduit à l'identique, en plus concis** : la méthodologie d'évaluation du coût (§4.2) — trois opérateurs Flink dédiés et un *connected stream* deviennent trois lignes de `broadcast + treeReduce`.
- **Se traduit, avec un degré de liberté nouveau** : le parallélisme. Le couplage partitions/cœurs de Flink se découple côté Spark (§3.5), ce qui permet une expérience (E3bis) que le modèle de Flink ne permet pas.
- **Ne se traduit pas** : Queryable State (§4.3). C'est la seule perte de fonctionnalité nette identifiée dans ce portage — discutée au chapitre 6.

Chapitre 5

Évaluation expérimentale

Conformément à notre méthodologie de travail (chapitre 1), la qualité du clustering est validée *avant* sa scalabilité : mesurer un speedup sur un algorithme dont le résultat serait faux n’aurait aucun sens.

5.1 Protocole et environnement

5.1.1 Optimisation du kernel local (JMH)

Travail mené en parallèle sur les couches `core/coreset` uniquement (Structure of Arrays, tableaux primitifs, suppression du boxing — `streamkm.core.PointsSoA`), mesuré avec JMH plutôt que dans un job Spark : le JMH mesure une JVM isolée, il ne peut rien dire d’un job distribué. Les deux mesures (kernel local puis job Spark de bout en bout, §??-??) répondent à deux questions différentes.

Hypothèse. La représentation initiale (*Array of Structures* : `Array[Point]`, chaque `Point` portant son propre `Array[Double]`) impose une double indirection à chaque accès aux coordonnées, exactement le *pointer chasing* que la littérature sur la disposition mémoire identifie comme coûteux en cache. L’hypothèse de départ était qu’une représentation *Structure of Arrays* hors-tas (`PointsSoA` : un unique `java.nio.DoubleBuffer` adossé à `ByteBuffer.allocateDirect`, coordonnées contiguës en *row-major*) réduirait ce coût sur les boucles les plus critiques du pipeline (`Distance.sqdist`, appelée à chaque affectation de point dans `lloyd` et à chaque descente dans `CoresetTree`).

Protocole. JMH 1.37, 5 itérations de warmup, 30 itérations de mesure, 1 fork, modes `Throughput` et `AverageTime`, JDK identique pour les deux versions comparées (condition nécessaire à la validité de la comparaison). Six benchmarks isolent chacun une brique du pipeline : `sqdist` et `nearest` (calcul pur, sans allocation ni GC), `CoresetTree.build` et `BucketManager.insertAll` (écriture/lecture de buffers, avec allocation d’objets `Node`), `KMeans.lloyd` et le pipeline complet `fullPipeline` (opérations composites, mêlant accès aux points et calcul indépendant de leur représentation).

Résultat — l’hypothèse est infirmée. Contrairement à l’attente, la version SoA hors-tas est **systématiquement plus lente** que l’AoS d’origine, sur les six benchmarks et les deux modes de mesure, sans exception :

Analyse. Le profil de l’écart est le résultat le plus instructif : il est **maximal sur les opérations les plus pures** (`sqdist`, sans aucune allocation ni interaction avec le GC en jeu) et **s’atténue** à mesure que l’opération mesurée mêle davantage de calcul indépendant de la représentation

Benchmark	Nature de l'opération	Écart SoA vs AoS (avgt)
<code>sqdist</code>	calcul pur	+46 à +77 %
<code>nearest</code>	calcul pur	+45 à +51 %
<code>insertAll</code>	écriture de buffer	+32 à +50 %
<code>CoresetTree.build</code>	mixte (buffer + alloc.)	+36 à +59 %
<code>lloyd</code>	mixte	+20 à +30 %
<code>fullPipeline</code>	composite	+15 à +21 %

TABLE 5.1 – Écart relatif de latence moyenne (`AverageTime`), SoA hors-tas vs AoS, sur les six benchmarks du kernel local.

des points (accumulation dans des tableaux `Array[Double]` on-heap pour `lloyd`, redémarrages *k*-means++ et construction d'arbre pour `fullPipeline`). Cette gradation pointe vers une cause structurelle unique, localisée dans l'accès lui-même : `java.nio.DoubleBuffer.get/put` effectue une vérification de bornes *supplémentaire* par rapport à celle déjà faite par la JVM sur un `Array`, et ne bénéficie pas des mêmes intrinsics HotSpot (élimination de bornes-checks prouvée statiquement, auto-vectorisation SIMD) que des décennies d'ingénierie ont concentrées sur les tableaux primitifs. Le bénéfice théorique de localité mémoire du SoA — réel en principe — est ici *dominé* par le coût d'indirection de l'API `Buffer` choisie pour le matérialiser.

Ce que ce résultat ne remet pas en cause, et piste corrective. Ce n'est pas le principe SoA qui est invalidé, mais le choix précis d'implémentation (`ByteBuffer.allocateDirect` + `DoubleBuffer` plutôt qu'un `Array[Double]` plat on-heap). Une variante SoA *on-heap* capturerait la localité mémoire sans l'overhead d'indirection identifié ci-dessus, au prix de l'abandon du gain espéré sur la pression GC (hors-tas). Une comparaison à trois branches (AoS / SoA on-heap / SoA hors-tas) permettrait de décomposer précisément la contribution de chacun des deux axes pris séparément — non menée dans le temps imparti à ce projet.

Chapitre 6

Discussion — points forts et points faibles

6.1 Points forts

Un seul passage sur les données, mémoire bornée. Comme la thèse, notre implémentation ne lit chaque point qu’une seule fois et ne conserve qu’un résumé de taille $O(m \cdot \log(n/m))$ — jamais l’ensemble du flux (chapitre 2, §2.4). C’est la propriété qui rend StreamKM++ applicable à des flux dont la taille finale est inconnue à l’avance, contrairement à un k -means batch classique.

La fusion arborescente lève le principal goulot identifié par la thèse elle-même. La thèse repère et documente (§5.3.1) que son opérateur `FlatMapToFinalCoreset` plafonne le speedup au-delà de 8 cœurs parce qu’il est non-parallèle par construction. Notre `treeReduce` (chapitre 4, §4.1) répond directement à cette limite — l’expérience E7 (chapitre 5) est conçue spécifiquement pour vérifier que ce plafond recule effectivement.

Simplifications nettes permises par le modèle micro-batch de Spark. La disparition complète de `HandleWatermark` (§4.4) et la réduction de la méthodologie d’évaluation du coût à trois lignes de `broadcast` + `treeReduce` (§4.2) sont deux exemples concrets où le portage n’est pas seulement possible, mais strictement plus simple que l’original Flink.

Un degré de liberté expérimental que Flink n’offre pas. Le découplage partitions/cœurs (§3.5) permet de tester directement l’explication que la thèse avance, sans pouvoir la vérifier, pour la baisse de coût observée quand le parallélisme augmente (§5.3.1, fig. 5.9) — notre expérience E3bis comble cette lacune.

Architecture qui protège la correction algorithmique du code de distribution. La séparation stricte `core/coreset/spark` (chapitre 1) a permis de valider entièrement la logique de StreamKM++ (chapitre 2, 21 tests) *avant* d’écrire une seule ligne de code Spark, et de découvrir qu’aucune extension de `streamkm.coreset` n’était nécessaire pour distribuer merge-and-reduce (chapitre 4) — signe que la frontière entre les couches avait été bien placée dès la conception.

6.2 Points faibles

Le clustering final reste un goulot non parallèle. Comme chez Flink, l’application finale de k -means++ sur le coreset global (`KMeans.fit`) tourne entièrement sur le *driver* — un temps constant qui, à mesure que le nombre de cœurs augmente, finit par dominer le temps total et

plafonner le speedup (exactement ce qu’observe la thèse au-delà de 8 cœurs, §5.3.1). Le portage vers Spark ne résout pas ce problème structurel de l’algorithme lui-même ; il ne fait que repousser l’autre goulot (la fusion des coresets partiels, §4.1).

Latence structurelle du modèle micro-batch. Même avec un `Trigger` agressif, Spark Structured Streaming ne peut pas égaler la latence d’un moteur de vrai streaming comme Flink : un point n’est traité qu’à la clôture de son micro-batch, jamais immédiatement à son arrivée (chapitre 3).

Absence d’équivalent à Queryable State. Flink permet d’interroger l’état interne d’un job en cours d’exécution depuis l’extérieur, sans latence ajoutée. Spark n’expose aucune primitive équivalente ; le contournement usuel (sink `memory` + requête SQL, ou écriture externe périodique) n’offre pas la même garantie de fraîcheur (§4.3). C’est la seule perte de fonctionnalité nette identifiée dans ce portage.

Aucune gestion du *concept drift*. Comme dans la thèse, les anciens buckets ne sont jamais oubliés — un point vu au tout début du flux pèse dans le coreset final exactement comme un point vu à l’instant présent. Pour un flux dont la distribution change réellement au cours du temps, ce n’est pas nécessairement souhaitable, et StreamKM++ n’offre aucun mécanisme d’oubli (contrairement, par exemple, au facteur de décroissance de `StreamingKMeans` — cf. paragraphe suivant).

Sensibilité à l’initialisation des alternatives non garanties. La comparaison à `StreamingKMeans` (chapitre 5, §??) illustre concrètement ce que la garantie théorique de coreset apporte en pratique : sans cette garantie, un mauvais tirage d’initialisation peut dégrader le résultat de plusieurs ordres de grandeur, sans qu’aucun signal ne le révèle en cours d’exécution. StreamKM++ n’est pas à l’abri d’un coreset de mauvaise qualité, mais la conservation de la masse et la borne théorique sur l’erreur (chapitre 2) donnent au moins un signal de correction vérifiable — ce que `StreamingKMeans` n’offre pas.

Non-déterminisme lié au partitionnement. Le résultat dépend du nombre de partitions choisi (pas seulement du nombre de cœurs), puisque chaque partition construit son propre coreset intermédiaire avec sa propre graine (§4.1). Nous fixons les graines par partition pour la reproductibilité à *nombre de partitions fixé*, mais deux exécutions avec des partitionnements différents produisent, légitimement, des coresets différents.

Aucune standardisation des features. Les expériences sur Coverttype (chapitre 5) utilisent les données brutes, comme le fait la thèse. Les colonnes one-hot (échelle $\{0, 1\}$) et les mesures brutes (échelle jusqu’à $\sim 7\,000$) pèsent alors très différemment dans une distance euclidienne au carré — une limite connue et documentée dans le code, pas corrigée pour rester fidèle au protocole de la thèse et pour ne pas avantager artificiellement une des méthodes comparées à l’expérience E6.

Chapitre 7

Conclusion et perspectives

Ce rapport a décrit le portage de StreamKM++ depuis l’implémentation Flink de Bitsakis (2018) vers Apache Spark. Les trois briques algorithmiques de la thèse — Lloyd pondéré, seeding k -means++, coresets tree et merge-and-reduce — ont été implémentées et validées indépendamment de tout moteur de calcul distribué (chapitre 2), avant d’être orchestrées par une couche Spark qui ne contient elle-même aucune logique algorithmique propre (chapitre 4). Cet ordre — théorie, puis framework, puis implémentation opérateur par opérateur, puis validation de la qualité avant la scalabilité — a été tenu tout au long du projet, et a permis de découvrir et corriger un bug réel (une graine aléatoire mal propagée dans la fusion de coresets) avant qu’il n’affecte les expériences du chapitre 5.

Le portage confirme ce que l’énoncé posait comme question implicite : une partie du dataflow de la thèse se traduit directement (voire se simplifie, comme la disparition de `HandleWatermark` ou la méthodologie d’évaluation du coût), une partie se traduit avec un gain net (la fusion arborescente des coresets, qui répond directement au goulot que la thèse identifie elle-même), et une partie ne se traduit pas du tout (Queryable State) — sans que cela remette en cause la viabilité du portage dans son ensemble (chapitre 6).

7.1 Travaux futurs

Variante à requêtes périodiques (§4.3 de la thèse). Décrite mais non implémentée faute de temps (chapitre 4, §4.3) — le chemin d’implémentation est cependant déjà balisé : `Trigger.ProcessingTime` pour la ré-évaluation périodique, `BucketManager.extractCoreset` (qui ne vide jamais l’état) pour dupliquer le coresets courant à plusieurs valeurs de k .

Un vrai équivalent de Queryable State. La seule perte de fonctionnalité nette identifiée dans ce portage (chapitre 6). Une piste non explorée : un service externe léger interrogeant directement l’état exposé par `onModel`, plutôt qu’un sink `memory + SQL` — à évaluer en termes de fraîcheur et de complexité ajoutée.

Gestion du concept drift. Ajouter un mécanisme d’oubli (fenêtre glissante sur les buckets les plus anciens, ou facteur de décroissance à la `StreamingKMeans`) permettrait de traiter des flux dont la distribution change réellement dans le temps — hors du périmètre de StreamKM++ tel que défini par la thèse, mais une extension naturelle.

Poursuivre l’optimisation du kernel local. Le travail Structure-of-Arrays / hors-tas engagé sur `core/coreset` (chapitre 5, benchmarks JMH) ouvre la voie à des optimisations supplémentaires une fois les résultats consolidés — en particulier sur `CoresetTree.build`, identifié dès la spécification initiale comme le point chaud principal du pipeline.

.1 Code source complet

Cette annexe inclut, via `\lstinputlisting`, le contenu **réel** des fichiers du dépôt `StreamKMpp` au moment de la rédaction de ce rapport (dossier frère `../StreamKMpp` sur la même machine). Aucun extrait n'est retapé ou reconstitué de mémoire.

.1.1 Configuration du projet

```
1 // Scala 2.12 est imposé par Spark 3.5 (E4). On s'y tient dès maintenant pour éviter
2 // une migration au moment d'ajouter la couche Spark.
3 ThisBuild / scalaVersion := "2.12.18"
4 ThisBuild / organization := "streamkm"
5 ThisBuild / version      := "0.1.0-SNAPSHOT"
6
7 // Spark 3.5.x : dernière branche 3.5 supportant nativement Java 17 (E4).
8 // Scope "provided" : disponible en compile/test, absent du jar assemblé - c'est
9 // spark-submit qui fournit ces classes au runtime en production.
10 val sparkVersion = "3.5.3"
11
12 lazy val bench = (project in file("bench"))
13   .enablePlugins(JmhPlugin)
14   .dependsOn(root)
15   .settings(
16     scalaVersion := "2.12.18",
17     name         := "streamkm-bench"
18   )
19
20 lazy val root = (project in file("."))
21   .settings(
22     name := "streamkm-spark",
23     libraryDependencies ++= Seq(
24       "org.scalatest" %% "scalatest" % "3.2.18" % Test,
25       "org.apache.spark" %% "spark-core" % sparkVersion % Provided,
26       "org.apache.spark" %% "spark-sql" % sparkVersion % Provided,
27       // Ajouté pour streamkm.experiments.QualityMetrics : sert de référence externe
28       // (org.apache.spark.ml.clustering.KMeans, entraîné sur le dataset complet, pas
29       // sur un coreset) dans l'expérience de comparaison à MLlib.
30       "org.apache.spark" %% "spark-mllib" % sparkVersion % Provided
31     ),
32     scalacOptions ++= Seq(
33       "-deprecation",
34       "-feature",
35       "-unchecked",
36       "-Xlint"
37     ),
38     // -ea active les 'assert' d'invariants du BucketManager pendant les tests.
39     // En production (E4) on laisse -ea désactivé : les assertions coûtent cher
40     // dans les boucles chaudes.
41     //
42     // Les --add-opens sont indispensables pour faire tourner Spark sous Java 17 :
43     // Spark/Hadoop utilisent de la réflexion sur des modules internes du JDK
44     // (java.lang, java.nio, sun.nio.ch...) que le module system de Java 17 bloque
45     // par défaut (InaccessibleObjectException sans ce flag). Ce n'est PAS une
46     // fantaisie de config : sans ça, 'sbt test' plante dès la création du
47     // SparkContext. Spark lui-même ajoute ces mêmes flags dans ses scripts de
48     // lancement (spark-submit) à partir de la 3.3.
49     Test / fork := true,
50     Test / parallelExecution := false,
51     Test / javaHome := Some(file("/Users/victoriengb/Library/Java/
52       JavaVirtualMachines/temurin-21.0.11/Contents/Home")),
53     Test / javaOptions ++= Seq(
54       "-Xmx2g",
55       "-ea",
56       "--add-opens=java.base/java.lang=ALL-UNNAMED",
57       "--add-opens=java.base/java.lang.invoke=ALL-UNNAMED",
58       "--add-opens=java.base/java.lang.reflect=ALL-UNNAMED",
59       "--add-opens=java.base/java.io=ALL-UNNAMED",
60       "--add-opens=java.base/java.net=ALL-UNNAMED",
61       "--add-opens=java.base/java.nio=ALL-UNNAMED",
62       "--add-opens=java.base/java.util=ALL-UNNAMED",
```

```

62     "--add-opens=java.base/java.util.concurrent=ALL-UNNAMED",
63     "--add-opens=java.base/java.util.concurrent.atomic=ALL-UNNAMED",
64     "--add-opens=java.base/sun.nio.ch=ALL-UNNAMED",
65     "--add-opens=java.base/sun.nio.cs=ALL-UNNAMED",
66     "--add-opens=java.base/sun.security.action=ALL-UNNAMED",
67     "--add-opens=java.base/sun.util.calendar=ALL-UNNAMED"
68 ),
69
70 // 'run'/'runMain' : MÊME JDK que Test (Temurin 21), pas seulement les mêmes
71 // flags - sans ce 'javaHome', le process forké utilise le JDK système par
72 // défaut, dont le module system durci peut se comporter différemment vis-à-vis
73 // de '--add-opens' (constaté le 2026-08-27 : IllegalAccessError malgré le flag
74 // présent, disparu une fois le JDK aligné sur Temurin 21).
75 Compile / run / javaHome := (Test / javaHome).value,
76 Compile / run / fork      := true,
77 Compile / run / javaOptions := (Test / javaOptions).value.map {
78     case "-Xmx2g" => "-Xmx4g"
79     case other    => other
80 },
81
82 // Limitation sbt connue (cf. CLAUDE.md, note sous "Commandes") : par défaut,
83 // 'Compile / run' ET 'Compile / runMain' n'exposent PAS les dépendances
84 // "Provided" (Spark) sur leur classpath runtime - seul 'Test / *' les voit.
85 // 'run' et 'runMain' sont deux tâches sbt DISTINCTES (découvert le
86 // 2026-08-22 : corriger seulement 'run' ne suffit pas pour
87 // 'sbt runMain ...') ; les deux sont donc réalignées sur
88 // 'Compile / fullClasspath' (qui inclut Provided), pour que 'sbt run' ET
89 // 'sbt runMain streamkm.experiments.QualityMetrics' fonctionnent directement
90 // sans devoir passer par 'Test/runMain'.
91 Compile / run := Defaults.runTask(
92     Compile / fullClasspath,
93     Compile / run / mainClass,
94     Compile / run / runner
95 ).evaluated,
96 Compile / runMain := Defaults.runMainTask(
97     Compile / fullClasspath,
98     Compile / run / runner
99 ).evaluated
100 )

```

Listing 1 – build.sbt

1.1.2 Couche core : structures de données et k-means

```

1 package streamkm.core
2
3 /**
4  * Un point pondéré de l'espace euclidien de dimension d.
5  *
6  * 'weight' vaut 1.0 pour un point brut du flux, et la masse totale du sous-
7  *   ensemble
8  * représenté pour un point issu d'un coreset (thèse §2.3.3 : « we denote the
9  *   weight of
10  * each q_i as the total number of points in each subset P »).
11  *
12  * ATTENTION : 'coords' n'est pas copié défensivement, ni à la construction ni à
13  *   la
14  * lecture. C'est un choix délibéré - copier à chaque accès tuerait les
15  *   performances dans
16  * les boucles de 'Distance' et de 'CoresetTree'. La contrepartie est un
17  *   invariant à
18  * respecter partout : **un tableau de coordonnées n'est jamais muté après
19  *   construction**.
20  *
21  * Toute mutation potentielle passe par '.clone()' explicite.
22  *
23  * Cette classe reste volontairement une classe « objet » (Array of Structures)
24  *   en E1/E2.

```

```

17  * L'étape E6 introduira une représentation Structure of Arrays derrière la même
    API
18  * publique, et JMH mesurera l'écart.
19  */
20  final class Point(val coords: Array[Double], val weight: Double) extends
    Serializable {
21
22      def dim: Int = coords.length
23
24      /** Utile uniquement pour les messages d'erreur et le débogage. */
25      override def toString: String =
26          coords.mkString("Point([", ", ", s"]", w=$weight)")
27  }
28
29  object Point {
30
31      /** Point brut du flux : poids 1. */
32      def apply(coords: Array[Double]): Point = new Point(coords, 1.0)
33
34      def apply(coords: Array[Double], weight: Double): Point = new Point(coords,
          weight)
35
36      /** Confort pour les tests : 'Point(1.0, 2.0)'. */
37      def of(coords: Double*): Point = new Point(coords.toArray, 1.0)
38
39      /**
40       * Parsing d'une ligne CSV/SSV en Point.
41       * Renvoie None si la ligne est malformée ou n'a pas la dimension attendue - c
         'est
42       * l'équivalent du 'FlatMapToPoint' de la thèse (§4.1), qui utilise un flatMap
43       * précisément pour pouvoir rejeter les lignes invalides sans faire échouer le
         job.
44       */
45      def parse(line: String, dim: Int, sep: Char = ','): Option[Point] = {
46          val parts = line.trim.split(sep)
47          if (parts.length != dim) None
48          else {
49              val arr = new Array[Double](dim)
50              var i = 0
51              var ok = true
52              while (i < dim && ok) {
53                  try arr(i) = java.lang.Double.parseDouble(parts(i))
54                  catch { case _: NumberFormatException => ok = false }
55                  i += 1
56              }
57              if (ok) Some(new Point(arr, 1.0)) else None
58          }
59      }
60  }

```

Listing 2 – streamkm/core/Point.scala

```

1  package streamkm.core
2
3  import scala.util.Random
4
5  /** Résultat d'un clustering : les centres, la SSE pondérée atteinte, le nombre
    d'itérations. */
6  final case class KMeansModel(
7      centers:      PointsSoA,
8      cost:         Double,
9      iterations:  Int
10  ) {
11      def k: Int = centers.n

```

```

12     override def toString: String = s"KMeansModel(k=$k, cost=$cost, iterations=
13         $iterations)"
14 }
15 /**
16  * Échantillonnage pondéré, factorisé ici parce qu'il sert à trois endroits :
17  * le premier centre de k-means++, le premier représentant du coresets tree, et
18  * l'échantillonnage  $D^2$  à l'intérieur d'une feuille.
19  */
20 private[streamkm] object Sampling {
21
22     /**
23      * Tire un indice dans  $[0, n)$  avec une probabilité proportionnelle à  $w(i)$ .
24      * Si la masse totale est nulle (ou négative par erreur d'arrondi), retombe
25      sur
26      * un tirage uniforme plutôt que de lever une exception : ce cas se produit
27      * légitimement quand tous les points d'une feuille sont confondus.
28      */
29     def byWeight(n: Int, w: Int => Double, rng: Random): Int = {
30         var total = 0.0
31         var i = 0
32         while (i < n) { total += w(i); i += 1 }
33
34         if (!(total > 0.0)) return rng.nextInt(n)
35
36         val target = rng.nextDouble() * total
37         var acc = 0.0
38         i = 0
39         while (i < n) {
40             acc += w(i)
41             if (acc >= target) return i
42             i += 1
43         }
44         // Atteignable uniquement par accumulation d'erreurs flottantes.
45         n - 1
46     }
47 }
48 object KMeans {
49
50     /**
51      * Procédure de seeding de k-means++ (thèse, algorithme 2.2, lignes 1-3), **
52      version pondérée**.
53
54      * Différence assumée avec la thèse : la ligne 1 de l'algorithme 2.2 choisit
55      le premier
56      * centre « uniformly at random ». On tire ici proportionnellement au poids.
57      Sur des points
58      * bruts (tous de poids 1) les deux coïncident exactement ; sur un coresets (
59      poids très
60      * hétérogènes) le tirage uniforme biaiserait l'initialisation vers les ré
61      gions peu denses.
62      * Or dans StreamKM++, k-means++ est TOUJOURS appliqué à un coresets (§2.3.4, é
63      tape 2) -
64      * la version pondérée est donc la bonne. À mentionner dans le rapport.
65      *
66      * Coût :  $O(n \cdot k \cdot d)$ .
67      */
68     def seedPlusPlus(points: PointsSoA, k: Int, rng: Random): PointsSoA = {
69         require(points.n > 0, "seedPlusPlus: ensemble de points vide")
70         require(k > 0, s"seedPlusPlus: k doit être > 0 (reçu $k)")
71
72         val n = points.n

```



```

67     val kEff = math.min(k, n)
68     val centers = PointsSoA.allocate(kEff, points.dimension)
69     val tmp      = new Array[Double](points.dimension)
70
71     val i0 = Sampling.byWeight(n, points.weight, rng)
72     points.copyCoordinatesInto(i0, tmp)
73     centers.append(tmp, points.weight(i0))
74
75     // closest(i) = distance au carré de points(i) au plus proche centre déjà
       choisi.
76     val closest = new Array[Double](n)
77     var i = 0
78     while (i < n) { closest(i) = Distance.squareDistance(points, i, centers, 0);
       i += 1 }
79
80     var c = 1
81     while (c < kEff) {
82         val idx = Sampling.byWeight(n, j => points.weight(j) * closest(j), rng)
83         points.copyCoordinatesInto(idx, tmp)
84         centers.append(tmp, points.weight(idx))
85
86         i = 0
87         while (i < n) {
88             val d = Distance.squareDistance(points, i, centers, c)
89             if (d < closest(i)) closest(i) = d
90             i += 1
91         }
92         c += 1
93     }
94     centers
95 }
96
97 /**
98  * Algorithme de Lloyd pondéré (thèse, algorithme 2.1).
99  *
100  * Le centre d'un cluster est la moyenne PONDÉRÉE de ses points :
101  *  $\mu_c = \sum_{\{p \in c\}} w_p \cdot p / \sum_{\{p \in c\}} w_p$ 
102  * Oublier les poids ici est le bug classique de StreamKM++ : le code tourne,
       ne plante
103  * jamaï, et produit des centres silencieusement faux dès qu'on l'applique à
       un coreset.
104  *
105  * Clusters vides : on conserve le centre précédent. C'est le choix le plus
       simple et il
106  * ne dégrade pas le coût (un cluster vide contribue 0). Alternative classique
       - ré-amorcer
107  * sur le point le plus éloigné - écartée : elle rend le coût non monotone et
       casse
108  * l'assertion de convergence.
109  *
110  * Arrêt : décroissance relative du coût inférieure à 'tol', ou 'maxIter'
       atteint.
111  */
112 def lloyd(
113     points: PointsSoA,
114     init: PointsSoA,
115     maxIter: Int = 100,
116     tol: Double = 1e-6
117 ): KMeansModel = {
118     require(points.n > 0, "lloyd: ensemble de points vide")
119     require(init.n > 0, "lloyd: aucun centre initial")
120
121     val k = init.n

```

```

122 val dimension = points.dimension
123 val centers = PointsSoA.allocate(k, dimension)
124 val tmp     = new Array[Double](dimension)
125 var c = 0
126 while (c < k) { init.copyCoordinatesInto(c, tmp); centers.append(tmp, init.
    weight(c)); c += 1 }
127 init.free() // ← init entièrement copié dans centers, plus jamais utilisé
    après ce point
128
129 val clusterSums = Array.ofDim[Double](k, dimension)
130 val masses      = new Array[Double](k)
131 val nearestResult = new NearestResult(0, 0.0)
132
133 var prevCost = Double.MaxValue
134 var it      = 0
135 var done    = false
136
137 while (!done && it < maxIter) {
138     c = 0
139     while (c < k) {
140         java.util.Arrays.fill(clusterSums(c), 0.0)
141         masses(c) = 0.0
142         c += 1
143     }
144
145     var currCost = 0.0
146     var i = 0
147     while (i < points.n) {
148         Distance.nearest(points, i, centers, nearestResult)
149         val idx = nearestResult.index
150         val weight = points.weight(i)
151         currCost += weight * nearestResult.distance
152
153         val clusterSum = clusterSums(idx)
154         var j = 0
155         while (j < dimension) { clusterSum(j) += weight * points.coordinates(i,
            j); j += 1 }
156         masses(idx) += weight
157         i += 1
158     }
159
160     c = 0
161     while (c < k) {
162         if (masses(c) > 0.0) {
163             val clusterSum = clusterSums(c)
164             val mass      = masses(c)
165             var j = 0
166             while (j < dimension) { tmp(j) = clusterSum(j) / mass; j += 1 }
167             centers.set(c, tmp, centers.weight(c))
168         }
169         c += 1
170     }
171
172     it += 1
173     assert(currCost <= prevCost + 1e-9 * math.abs(prevCost),
174         s"Lloyd: coût non monotone ($prevCost -> $currCost)")
175
176     if (prevCost - currCost <= tol * math.max(math.abs(prevCost), 1e-300))
177         done = true
178     prevCost = currCost
179 }
180 KMeansModel(centers, cost(points, centers), it)

```

```

181 }
182
183 /**
184  * k-means++ complet : 'nRestarts' exécutions indépendantes (seeding + Lloyd),
185  *   on garde
186  * la moins coûteuse. La thèse et le papier original utilisent 5 (§2.3.4, é
187  *   tape 2), mais
188  * le §5.3.1 montre qu'une seule application suffit en pratique - c'est un
189  *   paramètre
190  * d'expérience (E3), pas une constante.
191  */
192 def fit(
193     points: PointsSoA,
194     k: Int,
195     nRestarts: Int = 5,
196     maxIter: Int = 100,
197     tol: Double = 1e-6,
198     seed: Long = 42L
199 ): KMeansModel = {
200     require(nRestarts > 0, s"fit: nRestarts doit être > 0 (reçu $nRestarts)")
201     val rng = new Random(seed)
202     var best: KMeansModel = null
203     var r = 0
204     while (r < nRestarts) {
205         val model = lloyd(points, seedPlusPlus(points, k, rng), maxIter, tol)
206         if (best == null || model.cost < best.cost) best = model
207         r += 1
208     }
209     best
210 }
211
212 /**
213  * Fonction objectif : SSE pondérée,  $\sum_p w_p \cdot \min_c \|p - c\|^2$ .
214  * C'est la mesure de qualité unique de tout le projet (thèse §4.2) : tous les
215  * chapitres expérimentaux comparent des valeurs produites par cette fonction.
216  */
217 def cost(points: PointsSoA, centers: PointsSoA): Double = {
218     val nearestResult = new NearestResult(0, 0.0)
219     var totalCost = 0.0
220     var i = 0
221     while (i < points.n) {
222         Distance.nearest(points, i, centers, nearestResult)
223         totalCost += points.weight(i) * nearestResult.distance
224         i += 1
225     }
226     totalCost
227 }

```

Listing 3 – streamkm/core/KMeans.scala

.1.3 Couche coresset : coresset tree et merge-and-reduce

```

1 package streamkm.coreset
2
3 import scala.collection.mutable.ArrayBuffer
4 import scala.util.Random
5 import streamkm.core.{PointsSoA, Distance, Sampling}
6
7 /**
8  * Nœud de l'arbre de coresset.
9  *

```

```

10 * NOTE DE NOMMAGE - importante, à reprendre dans le rapport.
11 * La thèse appelle 'weight(v)' l'attribut servant à la descente dans l'arbre,
    et le définit
12 * pour une feuille comme 'cost(P_v, q_v)', c'est-à-dire une SOMME DE DISTANCES
    AU CARRÉ.
13 * Mais elle appelle aussi « weight » le poids des points du coreset produit,
    qui vaut le
14 * NOMBRE (la masse) de points représentés. Ce sont deux quantités totalement
    différentes
15 * portant le même nom. On les sépare explicitement :
16 *   - 'cost' :  $\sum_{p \in P_v} w_p \cdot \|p - q_v\|^2$  → sert à la descente (étape 1 du
    §2.3.3)
17 *   - 'mass' :  $\sum_{p \in P_v} w_p$  → sert au poids du point de
    coreset en sortie
18 *
19 * @param points feuille : les points du cluster. Nœud interne : null.
20 * @param repr point représentatif q_v (stocké aussi sur les nœuds internes,
    il devient
21 *           le représentant du fils gauche lors d'une scission).
22 */
23 private[coreset] final class Node(
24   var pointIndices: Array[Int], //indices in set of PointsSoA
25   var representativeIndex: Int,
26   var cost: Double,
27   var mass: Double,
28   var left: Node,
29   var right: Node
30 ) {
31   def isLeaf: Boolean = left == null
32 }
33
34 /**
35  * Construction du coreset par arbre binaire de clustering divisif hiérarchique
36  * (thèse §2.3.3 « Coreset Tree » et algorithme 2.4).
37  *
38  * Complexité :  $O(m \cdot n)$  dans le pire cas, mais  $O(n \cdot \log m)$  en pratique, contre
     $O(n \cdot m)$ 
39  * pour l'AdaptiveCoreset de l'algorithme 2.3 - c'est toute la raison d'être de
    cette
40  * structure.
41  */
42 object CoresetTree {
43
44   /**
45    * Construit un coreset d'au plus 'm' points à partir de 'store' (le store
    ENTIER
46    * est réduit - pour réduire un sous-ensemble, le caller doit d'abord
    construire
47    * un 'PointsSoA' de ce sous-ensemble, cf. 'PointsSoA.concat' dans
    BucketManager).
48    *
49    * Le store 'store' passé en entrée est CONSOMMÉ : cette fonction en prend
    possession
50    * et appelle 'store.free()' avant de retourner, une fois les représentants
    copiés
51    * dans le store de sortie. Le caller ne doit plus l'utiliser après l'appel.
52    */
53   def build(setOfPoints: PointsSoA, coresetSize: Int, range: Random): PointsSoA
    = {
54     require(coresetSize > 0, s"build: coresetSize doit être > 0 (reçu
    $coresetSize)")
55     if (setOfPoints.n <= coresetSize){
56       return setOfPoints

```

```

57     }
58
59     val allIndices = Array.range(0, setOfPoints.n)
60     val rootRepresentativeIndex = Sampling.byWeight(setOfPoints.n, setOfPoints.
        weight, range)
61     val root = new Node(allIndices, rootRepresentativeIndex, 0.0, 0.0, null,
        null)
62     recomputeLeaf(root, setOfPoints)
63
64     var leafCount = 1
65     val pathFromRoot = new ArrayBuffer[Node](32)
66
67     while (leafCount < coresetsize && root.cost > 0.0) {
68         pathFromRoot.clear()
69         val chosenLeaf = descend(root, range, pathFromRoot)
70         val newRepresentativeIndex = sampleD2(chosenLeaf, setOfPoints, range)
71
72         if (newRepresentativeIndex < 0){
73             val result = collectLeaves(root, setOfPoints, coresetsize)
74             setOfPoints.free()
75             return result
76         }
77
78         if (!split(chosenLeaf, newRepresentativeIndex, setOfPoints)){
79             val result = collectLeaves(root, setOfPoints, coresetsize)
80             setOfPoints.free()
81             return result
82         }
83         var pathIndex = pathFromRoot.length - 2
84         while (pathIndex >= 0){
85             recomputeInner(pathFromRoot(pathIndex))
86             pathIndex -= 1
87         }
88         leafCount += 1
89     }
90
91     val result = collectLeaves(root, setOfPoints, coresetsize)
92     setOfPoints.free()
93     result
94 }
95
96 // -----
97 // Étapes internes (numérotées comme au §2.3.3)
98 // -----
99
100 /**
101  * Étape 1 : choisir une feuille avec une probabilité proportionnelle à son co
    ût.
102  *
103  * [!] AMBIGUÏTÉ DE LA SOURCE, tranchée ici et à signaler dans le rapport.
104  * Le §2.3.3 étape 1 dit « with a probability proportional to cost( $P_\ell$ ,  $q_\ell$ ) »
    , et détaille
105  * une descente où un fils est choisi avec probabilité  $\text{cost}(\text{fils})/\text{cost}(\text{père})$ .
    La ligne 5 de
106  * l'algorithme 2.4 dit en revanche « according to their weights ». Les deux
    formulations
107  * s'accordent si l'on comprend « weight( $v$ ) » au sens de la définition donnée
    juste avant
108  * l'algorithme (weight d'une feuille =  $\text{cost}(P_v, q_v)$ ) - ce que confirme le
    papier original
109  * d'Ackermann et al. C'est donc bien le COÛT qui pilote la descente.
110  *

```

```

111  * Le raisonnement le confirme : une feuille déjà bien représentée (coût
112    faible) ne doit
113  * plus être scindée ; l'effort de raffinement doit aller là où l'erreur est
    grande. Utiliser
114  * la masse à la place ferait dériver l'algorithme vers un simple é
   chantillonnage
115  * proportionnel à la densité, et lui ferait perdre la propriété de coresé.
116  *
117  * 'path' reçoit le chemin racine → feuille, pour la remontée.
118  */
119 private def descend(root: Node, range: Random, pathFromRoot: ArrayBuffer[Node
    ]): Node = {
120     var currentNode = root
121     pathFromRoot += currentNode
122
123     while (!currentNode.isLeaf) {
124         val totalCost = currentNode.cost
125         currentNode =
126             if (!(totalCost > 0.0))
127                 currentNode.left // sécurité : coût nul, choix arbitraire
128             else if (range.nextDouble() * totalCost < currentNode.left.cost)
129                 currentNode.left
130             else
131                 currentNode.right
132         pathFromRoot += currentNode
133     }
134     currentNode
135 }
136
137 /**
138  * Étape 2 : dans la feuille, tirer le prochain représentant avec une
    probabilité
139  * proportionnelle à sa distance au carré au représentant courant (é
    chantillonnage  $D^2$ ).
140  * Renvoie null si la feuille est de coût nul.
141  */
142 private def sampleD2(leaf: Node, setOfPoints: PointsSoA, rng: Random): Int = {
143     if (!(leaf.cost > 0.0)) return -1
144     val indices = leaf.pointIndices
145     val pickedPosition = Sampling.byWeight(
146         indices.length,
147         position => setOfPoints.weight(indices(position)) * Distance.
            squareDistance(setOfPoints, indices(position), leaf.
                representativeIndex),
148         rng
149     )
150     val chosenIndex = indices(pickedPosition)
151     if (Distance.squareDistance(setOfPoints, chosenIndex, leaf.
        representativeIndex) <= 0.0) -1 else chosenIndex
152 }
153
154 /**
155  * Étape 3 : scinder la feuille en deux fils selon la proximité à 'repr' et à
    'q',
156  * puis mettre à jour les attributs du nœud devenu interne.
157  *
158  * Les deux fils sont nécessairement non vides : 'repr' est à distance 0 de
    lui-même
159  * (donc à gauche) et 'q' est à distance 0 de lui-même et à distance > 0 de '
    repr'
160  * (donc à droite). Le 'false' de retour est une sécurité contre une violation
    de
    * cette propriété par erreur numérique.

```

```

161  */
162  private def split(leaf: Node, newRepresentativeIndex: Int, store: PointsSoA):
    Boolean = {
163      val indices      = leaf.pointIndices
164      val leftIndices  = new ArrayBuffer[Int](indices.length)
165      val rightIndices = new ArrayBuffer[Int](indices.length)
166
167      var position = 0
168      while (position < indices.length) {
169          val pointIndex = indices(position)
170          if (Distance.squareDistance(store, pointIndex, leaf.representativeIndex) <
              = Distance.squareDistance(store, pointIndex, newRepresentativeIndex))
171              leftIndices += pointIndex
172          else
173              rightIndices += pointIndex
174          position += 1
175      }
176      if (leftIndices.isEmpty || rightIndices.isEmpty) return false
177
178      val leftChild  = new Node(leftIndices.toArray, leaf.representativeIndex,
179                                0.0, 0.0, null, null)
180      val rightChild = new Node(rightIndices.toArray, newRepresentativeIndex,
181                                0.0, 0.0, null, null)
182      recomputeLeaf(leftChild, store); recomputeLeaf(rightChild, store)
183
184      leaf.pointIndices = null; leaf.left = leftChild; leaf.right = rightChild
185      recomputeInner(leaf)
186      true
187  }
188
189  private def recomputeLeaf(node: Node, setOfPoints: PointsSoA): Unit = {
190      var cost = 0.0; var mass = 0.0
191      val indices = node.pointIndices
192      var position = 0
193      while (position < indices.length) {
194          val pointIndex = indices(position)
195          cost += setOfPoints.weight(pointIndex) * Distance.squareDistance(
196              setOfPoints, pointIndex, node.representativeIndex)
197          mass += setOfPoints.weight(pointIndex)
198          position += 1
199      }
200      node.cost = cost; node.mass = mass
201  }
202
203  private def recomputeInner(node: Node): Unit = {
204      node.cost = node.left.cost + node.right.cost
205      node.mass = node.left.mass + node.right.mass
206  }
207
208  private def collectLeaves(root: Node, setOfPoints: PointsSoA, coresetsize: Int
209      ): PointsSoA = {
210      val output      = PointsSoA.allocate(coresetsize, setOfPoints.dimension)
211      val tmp          = new Array[Double](setOfPoints.dimension)
212      val nodeStack    = new ArrayBuffer[Node](); nodeStack += root
213      while (nodeStack.nonEmpty) {
214          val node = nodeStack.remove(nodeStack.length - 1)
215          if (node.isLeaf) { setOfPoints.copyCoordinatesInto(node.
216              representativeIndex, tmp); output.append(tmp, node.mass) }
217          else { nodeStack += node.left; nodeStack += node.right }
218      }
219      output
220  }
221  }

```

Listing 4 – streamkm/coreset/CoresetTree.scala

```

1 package streamkm.coreset
2
3 import scala.collection.mutable.ArrayBuffer
4 import scala.util.Random
5 import streamkm.core.PointsSoA
6
7 final class BucketManager(val coresetSize: Int, val dimension: Int, val seed:
8     Long) {
9     private val rng = new Random(seed)
10    // buckets(0) : tampon d'entrée, capacité coresetSize, rempli au fil de insert
11    ().
12    private val buckets: ArrayBuffer[PointsSoA] = ArrayBuffer(PointsSoA.allocate(
13        coresetSize, dimension))
14    private var totalWeightSeen: Double = 0.0
15    private var collapsed: Boolean = false
16
17    def insert(point: Array[Double], weight: Double): Unit = {
18        if (collapsed)
19            throw new IllegalStateException(
20                "BucketManager: insertion interdite après merge(). Un manager fusionné
21                est un " +
22                "résultat de combOp, il n'est plus alimenté par le flux."
23            )
24        buckets(0).append(point, weight)
25
26        totalWeightSeen += weight
27
28        if (buckets(0).n >= coresetSize) {
29            ensureLevel(1)
30            if (buckets(1).n == 0) {
31                // Déplacement pur : le contenu de buckets(0) devient buckets(1), sans r
32                éduction.
33                val movedBucket = buckets(0)
34                buckets(0) = PointsSoA.allocate(coresetSize, dimension)
35                buckets(1) = movedBucket
36            } else {
37                var pendingUnion: PointsSoA = PointsSoA.concat(buckets(0), buckets(1))
38                buckets(0).free(); buckets(0) = PointsSoA.allocate(coresetSize,
39                    dimension)
40                buckets(1).free(); buckets(1) = PointsSoA.allocate(coresetSize,
41                    dimension)
42
43                var level = 2
44                var placed = false
45                while (!placed) {
46                    val reducedCoreset = CoresetTree.build(pendingUnion, coresetSize, rng)
47                    // consomme et libère 'pendingUnion'
48                    ensureLevel(level)
49                    if (buckets(level).n == 0) { buckets(level) = reducedCoreset; placed =
50                        true }
51                    else {
52                        pendingUnion = PointsSoA.concat(reducedCoreset, buckets(level))
53                        reducedCoreset.free()
54                        buckets(level).free(); buckets(level) = PointsSoA.allocate(
55                            coresetSize, dimension)
56                        level += 1
57                    }
58                }
59            }
60        }
61    }
62}

```



```

51 }
52
53 /**
54  * Fusion de deux structures indépendantes - combOp pour un usage hors Spark
55  * (tests, démonstrateur). Repose sur les mêmes observations que la version
56  * AoS
57  * (thèse §2.3.4) : union de deux  $\epsilon$ -coresets disjoints =  $\epsilon$ -coreset de l'union
58  * ;
59  * composition multiplicative de l'erreur à chaque niveau. Graine combinée par
60  * XOR - même principe que StreamKMPlusPlus.mergeCoresets (indépendance de
61  * l'aléa à chaque étage de fusion, CLAUDE.md décision 10).
62  *
63  * Ne modifie ni 'this' ni 'other' - leurs stores propres restent intacts et
64  * libérables indépendamment par leurs appelants respectifs.
65  */
66 def merge(other: BucketManager): BucketManager = {
67     require(other.coresetSize == coresetSize, s"merge: tailles de coreset
68     incompatibles ($coresetSize vs ${other.coresetSize})")
69     require(other.dimension == dimension, s"merge: dimensions incompatibles (
70     $dimension vs ${other.dimension})")
71
72     val out = new BucketManager(coresetSize, dimension, seed ^ other.seed)
73
74     val thisUnion = union()
75     val otherUnion = other.union()
76     val combined = PointsSoA.concat(thisUnion, otherUnion)
77     thisUnion.free(); otherUnion.free()
78
79     val reduced = CoresetTree.build(combined, coresetSize, out.rng) // consomme
80     et libère 'combined'
81     out.ensureLevel(1)
82     out.buckets(1) = reduced
83     out.totalWeightSeen = totalWeightSeen + other.totalWeightSeen
84     out.collapsed = true
85     out
86 }
87
88 /**
89  * Union de tous les buckets non vides, SANS réduction. Renvoie TOUJOURS un
90  * store
91  * fraîchement alloué, jamais une référence directe vers un bucket vivant - mê
92  * me
93  * quand un seul bucket est non vide. C'est indispensable : 'extractCoreset()'
94  * passe
95  * ce résultat à 'CoresetTree.build', qui en prend possession (le libère, ou
96  * le
97  * renvoie tel quel si déjà assez petit). Si 'union()' renvoyait directement
98  * un
99  * bucket vivant dans le cas à un seul élément, l'appelant recevrait soit un
100  * store
101  * libéré par erreur (use-after-free), soit un alias que 'BucketManager' croit
102  * toujours posséder alors qu'il a changé de propriétaire.
103  */
104 def union(): PointsSoA = {
105     val nonEmptyBuckets = buckets.filter(_._n > 0)
106     require(nonEmptyBuckets.nonEmpty, "union: aucun bucket non vide")
107
108     var accumulated = copyOf(nonEmptyBuckets(0)) // copie dès le départ,
109     jamais d'alias
110     var bucketIndex = 1
111     while (bucketIndex < nonEmptyBuckets.length) {
112         val merged = PointsSoA.concat(accumulated, nonEmptyBuckets(bucketIndex))
113         accumulated.free()

```

```

102     accumulated = merged
103     bucketIndex += 1
104 }
105 accumulated
106 }
107
108 /** Copie indépendante d'un store - nécessaire pour que 'union()' ne renvoie
109     jamais
110     * un alias d'un bucket vivant. */
111 private def copyOf(store: PointsSoA): PointsSoA = {
112     val output = PointsSoA.allocate(store.n, store.dimension)
113     val tmp = new Array[Double](store.dimension)
114     var pointIndex = 0
115     while (pointIndex < store.n) {
116         store.copyCoordinatesInto(pointIndex, tmp)
117         output.append(tmp, store.weight(pointIndex))
118         pointIndex += 1
119     }
120     output
121 }
122
123 def extractCoreset(): PointsSoA = {
124     val unionStore = union()
125     CoresetTree.build(unionStore, coresetSize, rng) // consomme et libère '
126     unionStore '
127 }
128
129 private def ensureLevel(level: Int): Unit =
130     while (buckets.length <= level) buckets += PointsSoA.allocate(coresetSize,
131         dimension)
132
133 def numPointsSeen: Double = totalWeightSeen
134
135 def numLevels: Int = buckets.length
136
137 def free(): Unit = {
138     var level = 0
139     while (level < buckets.length) { buckets(level).free(); level += 1 }
140 }
141
142 /**
143     * Insertion en lot depuis un 'PointsSoA' déjà construit - évite l'aller-
144     retour
145     * PointsSoA → Array[Double] → BucketManager → PointsSoA interne qu'imposerait
146     * un simple appel répété à 'insert' depuis l'extérieur. Un seul tableau
147     tampon
148     * est réutilisé sur toute la boucle : aucune allocation par point.
149     */
150 def insertAll(store: PointsSoA): Unit = {
151     require(store.dimension == dimension, s"insertAll: dimension incompatible ($
152         {store.dimension} vs $dimension)")
153     val tmp = new Array[Double](dimension)
154     var i = 0
155     while (i < store.n) {
156         store.copyCoordinatesInto(i, tmp)
157         insert(tmp, store.weight(i))
158         i += 1
159     }
160 }
161
162 /**
163     * Vérification des invariants du §2.3.4. Utilisée sous 'assert' : coûteuse et
164     * désactivée en production (cf. build.sbt, -ea).

```

```

159  */
160  private[coreset] def checkInvariants(): Boolean = {
161    if (buckets(0).n > coresetSize) return false
162    var level = 1
163    while (level < buckets.length) {
164      val size = buckets(level).n
165      if (size != 0 && size != coresetSize) return false
166      level += 1
167    }
168    true
169  }
170 }
171
172 object BucketManager {
173   def recommendedM(k: Int): Int = 200 * k
174   def apply(coresetSize: Int, dimension: Int, seed: Long = 42L): BucketManager =
175     new BucketManager(coresetSize, dimension, seed)
176 }

```

Listing 5 – streamkm/coreset/BucketManager.scala

.1.4 Couche spark : portage distribué

```

1  package streamkm.spark
2
3  import scala.util.Random
4  import org.apache.spark.rdd.RDD
5  import org.apache.spark.sql.DataFrame
6  import org.apache.spark.sql.streaming.StreamingQuery
7
8  import streamkm.core.{KMeans, KMeansModel, Point, PointsSoA}
9  import streamkm.coreset.{BucketManager, CoresetTree}
10
11  final case class StreamKMParams(
12      k:          Int,
13      m:          Int,
14      dim:        Int,
15      nRestarts:  Int = 5,
16      seed:       Long = 42L
17  )
18
19  object StreamKMPlusPlus {
20
21    /**
22     * Un coreset partiel par PARTITION, accompagné de la graine qui a servi à le
23     * produire - nécessaire pour que 'mergeCoresets' puisse combiner les graines
24     * de
25     * façon déterministe à chaque niveau de fusion (voir plus bas).
26     */
27    def partialCoresets(rdd: RDD[Point], p: StreamKMParams): RDD[(Array[Point],
28      Long)] =
29      rdd.mapPartitionsWithIndex { (idx, it) =>
30        val partitionSeed = p.seed + idx
31        val bucketManager = new BucketManager(p.m, p.dim, partitionSeed)
32        while (it.hasNext) {
33          val point = it.next()
34          bucketManager.insert(point.coords, point.weight)
35        }
36        val partialCoreset = bucketManager.extractCoreset()
37        val result          = PointsSoA.toPoints(partialCoreset)
38        partialCoreset.free()
39        bucketManager.free()

```

```

38     Iterator.single((result, partitionSeed))
39 }
40
41 /**
42  * Fusion en arbre sur des '(Array[Point], Long)' - jamais de PointsSoA sur le
43  * réseau. À CHAQUE niveau de fusion, la graine du nœud résultant est
44  * 'seedA ^ seedB' (mêmes opérandes combinés que 'BucketManager.merge' en E2,
45  * CLAUDE.md décision 10) : ça restaure l'indépendance de l'aléa à chaque é
46  * tage
47  * de l'arbre de réduction, pas seulement au premier niveau partition →
48  * coreset
49  * partiel.
50 */
51 def mergeCoresets(rdd: RDD[Point], p: StreamKMPParams, depth: Int = 2): Array[
52   Point] = {
53   val partial = partialCoresets(rdd, p)
54   val (finalPoints, _) = partial.treeReduce(
55     (left, right) => {
56       val (pointsA, seedA) = left
57       val (pointsB, seedB) = right
58       val combinedSeed      = seedA ^ seedB
59
60       val storeA    = PointsSoA.fromPoints(pointsA)
61       val storeB    = PointsSoA.fromPoints(pointsB)
62       val combined  = PointsSoA.concat(storeA, storeB)
63       storeA.free(); storeB.free()
64
65       val reduced = CoresetTree.build(combined, p.m, new Random(combinedSeed))
66       val result  = PointsSoA.toPoints(reduced)
67       reduced.free()
68
69       (result, combinedSeed)
70     },
71     depth
72   )
73   finalPoints
74 }
75
76 def run(
77   source:      DataFrame,
78   p:           StreamKMPParams,
79   onModel:     KMeansModel => Unit,
80   featuresCol: String = "features"
81 ): StreamingQuery = {
82
83   val global = new BucketManager(p.m, p.dim, p.seed)
84
85   source.writeStream.foreachBatch { (batch: DataFrame, batchId: Long) =>
86     val t0 = System.nanoTime()
87
88     val points: RDD[Point] = batch.rdd.map { row =>
89       val coords = row.getAs[Seq[Double]](featuresCol).toArray
90       Point(coords)
91     }
92     val numPartitions = points.getNumPartitions
93
94     val partial = mergeCoresets(points, p.copy(seed = p.seed + batchId))
95     var i = 0
96     while (i < partial.length) { global.insert(partial(i).coords, partial(i).
97       weight); i += 1 }
98
99     val coreset = global.extractCoreset()
100     val model   = KMeans.fit(coreset, p.k, p.nRestarts, seed = p.seed)

```

```

97     coreset.free()
98     val elapsedMs = (System.nanoTime() - t0) / 1e6
99
100    // scalastyle:off println
101    println(
102        s"[StreamKM++] batch=$batchId partitions=$numPartitions m=${p.m} k=${p.k}
          " +
103        f"pointsSeen=${global.numPointsSeen}%.0f buildTimeMs=$elapsedMs%.1f
          cost=${model.cost}%.4g"
104    )
105    // scalastyle:on println
106
107    onModel(model)
108  }.start()
109  }
110 }

```

Listing 6 – streamkm/spark/StreamKMPlusPlus.scala

```

1 package streamkm.spark
2
3 import org.apache.spark.rdd.RDD
4
5 import streamkm.core.{KMeans, KMeansModel, Point, PointsSoA}
6
7 object CostEvaluator {
8
9   /**
10    * Coût total (SSE pondérée) de 'centers' sur l'ensemble du RDD 'points'.
11    *
12    * 'centers' (PointsSoA, off-heap) n'est pas sérialisable - même contrainte
13    *   que
14    * pour 'StreamKMPlusPlus.mergeCoresets' : conversion en 'Array[Point]' (petit
15    *   borné par k) AVANT le broadcast, reconstruction d'un 'PointsSoA' local à
16    *   chaque partition, libéré aussitôt le coût partiel calculé. 'centers' lui-même
17    *   n'est jamais libéré ici : cette fonction ne fait que le LIRE (via
18    *   'toPoints'), l'appelant en reste propriétaire - même convention que
19    *   'KMeans.cost'.
20    */
21   def cost(points: RDD[Point], centers: PointsSoA, depth: Int = 2): Double = {
22     val centersAsPoints = PointsSoA.toPoints(centers)
23     val bcCenters      = points.sparkContext.broadcast(centersAsPoints)
24
25     points
26     .mapPartitions { it =>
27       val partitionPoints = it.toArray
28       if (partitionPoints.isEmpty) Iterator.single(0.0)
29       else {
30         val pointsStore = PointsSoA.fromPoints(partitionPoints)
31         val centersStore = PointsSoA.fromPoints(bcCenters.value)
32         val partialCost = KMeans.cost(pointsStore, centersStore)
33         pointsStore.free()
34         centersStore.free()
35         Iterator.single(partialCost)
36       }
37     }
38     .treeReduce(_ + _, depth)
39   }
40
41   def cost(points: RDD[Point], model: KMeansModel): Double = cost(points, model.centers)
42 }

```

.1.5 Couche experiments : harnais de mesure de qualité

```

1 package streamkm.experiments
2
3 import java.io.{File, FileOutputStream, OutputStreamWriter, PrintWriter}
4 import java.nio.charset.StandardCharsets
5
6 import scala.io.Source
7
8 import org.apache.spark.ml.clustering.{KMeans => MLKMeans}
9 import org.apache.spark.ml.linalg.{SQLDataTypes, Vectors}
10 import org.apache.spark.rdd.RDD
11 import org.apache.spark.sql.{Row => SqlRow, SparkSession}
12 import org.apache.spark.sql.types.{StructField, StructType}
13
14 import streamkm.core.{KMeans, Point, PointsSoA}
15 import streamkm.coreset.BucketManager
16 import streamkm.spark.{CostEvaluator, StreamKMParams, StreamKMPlusPlus}
17
18 object QualityMetrics {
19
20   final case class ResultRow(dataset: String, method: String, k: Int, m: Int,
21     cost: Double)
22
23   def loadCoverttype(path: String, dim: Int = 54, maxPoints: Option[Int] = None):
24     Array[Point] = {
25     val src = Source.fromFile(path)
26     try {
27       val lines = maxPoints match {
28         case Some(n) => src.getLines().take(n)
29         case None     => src.getLines()
30       }
31       lines.flatMap { line =>
32         val cols = line.trim.split(",")
33         if (cols.length < dim) None
34         else Point.parse(cols.take(dim).mkString(","), dim, ',')
35       }.toArray
36     } finally src.close()
37   }
38
39   /**
40    * Référence séquentielle. 'points' reste 'Array[Point]' en entrée (interop
41    * avec
42    * 'loadCoverttype'/'MLlib) - converti en 'PointsSoA' localement, libéré en
43    * sortie
44    * de méthode, aucun 'PointsSoA' ne s'échappe de cette fonction.
45    */
46   def sequentialCost(points: Array[Point], k: Int, m: Int, dim: Int, seed: Long)
47     : Double = {
48     val pointsStore = PointsSoA.fromPoints(points)
49     val manager     = BucketManager(m, dim, seed)
50     manager.insertAll(pointsStore)
51
52     val coreset = manager.extractCoreset()
53     val model   = KMeans.fit(coreset, k, nRestarts = 5, seed = seed)
54     coreset.free() // consommé par fit (copié dans seedPlusPlus/lloyd), plus
55     utile après

```

```

51     val result = KMeans.cost(pointsStore, model.centers)
52
53     pointsStore.free()
54     model.centers.free()
55     manager.free()
56     result
57 }
58
59 /**
60  * Version distribuée. 'StreamKMParams' exige maintenant 'dim'. Le coreset
61  * renvoyé
62  * par 'mergeCoresets' est 'Array[Point]' (sérialisable, cf. adaptation de
63  * 'StreamKMPlusPlus') - reconverti en 'PointsSoA' ici pour 'KMeans.fit', qui
64  * EN
65  * DEVIENT propriétaire pour la durée de l'ajustement mais ne le libère pas
66  * lui-même
67  * (seul 'init', le second argument, est libéré par 'lloyd' - cf. décision
68  * prise
69  * lors de l'adaptation de 'KMeans'). D'où le '.free()' explicite ici après
70  * usage.
71  */
72 def distributedCost(rdd: RDD[Point], k: Int, m: Int, dim: Int, seed: Long,
73   numPartitions: Int): Double = {
74     val partitioned = rdd.repartition(numPartitions)
75
76     val coresetPoints = StreamKMPlusPlus.mergeCoresets(partitioned,
77       StreamKMParams(k = k, m = m, dim = dim, seed = seed))
78     val coresetStore = PointsSoA.fromPoints(coresetPoints)
79
80     val model = KMeans.fit(coresetStore, k, nRestarts = 5, seed = seed)
81     coresetStore.free()
82
83     val result = CostEvaluator.cost(partitioned, model.centers)
84     model.centers.free()
85     result
86 }
87
88 /** Inchangé - ne touche jamais à PointsSoA, reste sur Point.coords
89  * directement. */
90 def mllibCost(spark: SparkSession, points: Array[Point], k: Int, seed: Long,
91   numPartitions: Int): Double = {
92     val schema = StructType(Seq(StructField("features", SQLDataTypes.VectorType,
93       nullable = false)))
94     val rowRdd = spark.sparkContext
95       .parallelize(points, numPartitions)
96       .map(p => SqlRow(Vectors.dense(p.coords)))
97     val df = spark.createDataFrame(rowRdd, schema)
98
99     val model = new MLKMeans().setK(k).setSeed(seed).setFeaturesCol("features").
100       fit(df)
101     model.summary.trainingCost
102 }
103
104 def runKSweep(
105     spark: SparkSession,
106     datasetName: String,
107     points: Array[Point],
108     dim: Int,
109     ks: Seq[Int],
110     numPartitions: Int,
111     seed: Long
112 ): Seq[ResultRow] = {
113     val rdd = spark.sparkContext.parallelize(points, numPartitions)

```

```

103 ks.flatMap { k =>
104     val m = BucketManager.recommendedM(k)
105     println(s"[QualityMetrics] k-sweep : k=$k, m=$m ...")
106
107     val seq = sequentialCost(points, k, m, dim, seed)
108     val dist = distributedCost(rdd, k, m, dim, seed, numPartitions)
109     val mll = mllibCost(spark, points, k, seed, numPartitions)
110
111     Seq(
112         ResultRow(datasetName, "Séquentiel (E2)", k, m,
113             seq),
114         ResultRow(datasetName, "Spark distribué (notre implémentation)", k, m,
115             dist),
116         ResultRow(datasetName, "MLlib KMeans (batch, sans coresets)", k, m,
117             mll)
118     )
119 }
120
121 def runMSweep(
122     spark: SparkSession,
123     datasetName: String,
124     points: Array[Point],
125     dim: Int,
126     k: Int,
127     ms: Seq[Int],
128     numPartitions: Int,
129     seed: Long
130 ): Seq[ResultRow] = {
131     val rdd = spark.sparkContext.parallelize(points, numPartitions)
132     ms.flatMap { m =>
133         println(s"[QualityMetrics] m-sweep : m=$m (k=$k) ...")
134         val seq = sequentialCost(points, k, m, dim, seed)
135         val dist = distributedCost(rdd, k, m, dim, seed, numPartitions)
136         Seq(
137             ResultRow(datasetName, "Séquentiel (E2)", k, m,
138                 seq),
139             ResultRow(datasetName, "Spark distribué (notre implémentation)", k, m,
140                 dist)
141         )
142     }
143 }
144
145 def writeCsv(path: String, rows: Seq[ResultRow]): Unit = {
146     val file = new File(path)
147     Option(file.getParentFile).foreach(_.mkdirs())
148     val pw = new PrintWriter(new OutputStreamWriter(new FileOutputStream(file),
149         StandardCharsets.UTF_8))
150     try {
151         pw.println("dataset,method,k,m,cost")
152         rows.foreach(r => pw.println(s"${r.dataset},${r.method},${r.k},${r.m},${r.
153             cost}"))
154     } finally pw.close()
155 }
156
157 def main(args: Array[String]): Unit = {
158     val path = if (args.length > 0) args(0) else "data/covertime/
159         covtype.data"
160     val maxPoints = if (args.length > 1 && args(1).toInt > 0) Some(args(1).
161         toInt) else None
162     val dim = 54
163     val seed = 2026L
164     val numPartitions = 8

```



```

157
158 val spark = SparkSession.builder()
159   .appName("QualityMetrics")
160   .master("local[*]")
161   .config("spark.ui.enabled", "false")
162   .getOrCreate()
163
164 try {
165   println(s"[QualityMetrics] chargement de $path (maxPoints=$maxPoints)...")
166   val points = loadCovertypes(path, dim = dim, maxPoints = maxPoints)
167   println(s"[QualityMetrics] ${points.length} points chargés.")
168
169   val kSweepRows = runKSweep(spark, "Covertypes", points, dim, Seq(10, 20,
170     30, 40, 50), numPartitions, seed)
171   writeCsv("results/quality_k_sweep_covertypes.csv", kSweepRows)
172
173   val mSweepRows = runMSweep(spark, "Covertypes", points, dim, k = 25, Seq
174     (1000, 5000, 10000, 20000, 50000), numPartitions, seed)
175   writeCsv("results/quality_m_sweep_covertypes.csv", mSweepRows)
176
177   println("[QualityMetrics] terminé.")
178   println("  -> results/quality_k_sweep_covertypes.csv")
179   println("  -> results/quality_m_sweep_covertypes.csv")
180 } finally {
181   spark.stop()
182 }

```

Listing 8 – streamkm/experiments/QualityMetrics.scala

1.6 Fonction Main

Le main se lance avec :

```

1 sbt run # choisir streamkm.demo.Main si plusieurs mains sont proposes

```

```

1 package streamkm.demo
2
3 import scala.util.Random
4
5 import streamkm.core.{Distance, KMeans, NearestResult, Point, PointsSoA}
6 import streamkm.coreset.BucketManager
7
8 object Main {
9
10   private def gaussianMixture(
11     n: Int, k: Int, d: Int, sigma: Double, spread:
12       Double, rng: Random
13   ): (Array[Point], Array[Array[Double]]) = {
14     val trueCenters = Array.tabulate(k) { c =>
15       Array.tabulate(d)(j => ((c + 1) * (j + 1) % k).toDouble * spread)
16     }
17     val points = Array.tabulate(n) { _ =>
18       val ctr = trueCenters(rng.nextInt(k))
19       Point(Array.tabulate(d)(j => ctr(j) + rng.nextGaussian() * sigma))
20     }
21     (points, trueCenters)
22   }
23
24   def main(args: Array[String]): Unit = {
25     val n = 200000
26   }
27 }

```

```

25     val k      = 5
26     val d      = 2
27     val m      = BucketManager.recommendedM(k)
28     val seed    = 42L
29     val rng     = new Random(seed)
30
31     val (points, trueCenters) = gaussianMixture(n, k, d, sigma = 0.5, spread =
        20.0, rng)
32
33     println(s"StreamKM++ (démo E1/E2, SoA off-heap) - n=$n, k=$k, d=$d, m=$m")
34
35     val pointsStore = PointsSoA.fromPoints(points)
36
37     val t0          = System.nanoTime()
38     val manager     = BucketManager(m, d, seed)
39     manager.insertAll(pointsStore)
40     val tIngest     = (System.nanoTime() - t0) / 1e6
41
42     val t1          = System.nanoTime()
43     val coreset     = manager.extractCoreset()
44     val coresetSize = coreset.n
45     val tExtract    = (System.nanoTime() - t1) / 1e6
46
47     val t2          = System.nanoTime()
48     val model       = KMeans.fit(coreset, k, nRestarts = 5, seed = seed)
49     val tKmeans     = (System.nanoTime() - t2) / 1e6
50     coreset.free()
51
52     val costOnFullData = KMeans.cost(pointsStore, model.centers)
53     val batchModel     = KMeans.fit(pointsStore, k, nRestarts = 5, seed = seed)
54     val batchCost      = batchModel.cost
55
56     val trueCentersStore = PointsSoA.fromPoints(trueCenters.map(Point(_)))
57     val nearestResult    = new NearestResult(0, 0.0)
58     var maxDeviation     = 0.0
59     var i = 0
60     while (i < trueCentersStore.n) {
61         Distance.nearest(trueCentersStore, i, model.centers, nearestResult)
62         val deviation = math.sqrt(nearestResult.distance)
63         if (deviation > maxDeviation) maxDeviation = deviation
64         i += 1
65     }
66
67     println(f"Points vus par le manager : ${manager.numPointsSeen}%.0f (attendu
        $n)")
68     println(f"Coreset final                : $coresetSize points (m=$m)")
69     println(f"Temps ingestion flux         : $tIngest%.1f ms")
70     println(f"Temps extraction coreset     : $tExtract%.1f ms")
71     println(f"Temps k-means++ (coreset)    : $tKmeans%.1f ms")
72     println()
73     println(f"Coût (coreset) sur données complètes : $costOnFullData%.2f")
74     println(f"Coût k-means++ batch (référence)      : $batchCost%.2f (ratio ${
        costOnFullData / batchCost}%.3f)")
75     println(f"Écart max centre trouvé / vrai centre : $maxDeviation%.3f")
76     println()
77     println("Centres trouvés :")
78     val centerCoords = new Array[Double](d)
79     i = 0
80     while (i < model.centers.n) {
81         model.centers.copyCoordinatesInto(i, centerCoords)
82         println("  " + centerCoords.map(v => f"$v%.2f").mkString("(", ", ", ", ")"))
83         i += 1
84     }

```

```

85
86     pointsStore.free()
87     trueCentersStore.free()
88     model.centers.free()
89     batchModel.centers.free()
90     manager.free()
91 }
92 }

```

Listing 9 – streamkm/demo/Main.scala : démonstrateur section 1/section 2 sur données synthétiques

.1.7 Suite de tests

```

1  package streamkm
2
3  import scala.util.Random
4
5  import streamkm.core.{Distance, Point}
6
7  /**
8   * Générateur de données synthétiques à vérité terrain.
9   *
10  * C'est l'outil de l'expérience E1 (correction) : on connaît les vrais centres,
11  * donc on
12  * peut vérifier que l'algorithme les retrouve, ce qu'aucun jeu réel ne permet.
13  * Il servira
14  * aussi en E3/E4 pour faire varier la taille des données à volonté sans dé
15  * pendre d'un
16  * téléchargement.
17  */
18  object TestData {
19
20      final case class Mixture(points: Array[Point], trueCenters: Array[Array[Double]
21          ])
22
23      /**
24       * Mélange de 'k' gaussiennes isotropes de dimension 'd', dont les centres
25       * sont placés
26       * sur une grille de pas 'spread' pour garantir la séparabilité.
27       */
28      def gaussianMixture(
29          n:      Int,
30          k:      Int,
31          d:      Int,
32          sigma:  Double = 0.5,
33          spread: Double = 20.0,
34          seed:   Long   = 12345L
35      ): Mixture = {
36          val rng = new Random(seed)
37
38          val centers = Array.tabulate(k) { c =>
39              Array.tabulate(d)(j => ((c + 1) * (j + 1) % k).toDouble * spread)
40          }
41
42          val pts = Array.tabulate(n) { _ =>
43              val c = rng.nextInt(k)
44              val ctr = centers(c)
45              Point(Array.tabulate(d)(j => ctr(j) + rng.nextGaussian() * sigma))
46          }
47
48          Mixture(pts, centers)
49      }
50  }

```

```

44 }
45
46 /** Somme des poids d'un ensemble de points. */
47 def totalMass(pts: Array[Point]): Double = {
48     var s = 0.0
49     var i = 0
50     while (i < pts.length) { s += pts(i).weight; i += 1 }
51     s
52 }
53
54 /**
55  * Distance maximale entre chaque vrai centre et le centre estimé le plus
56    proche.
57  * Sert à vérifier qu'un clustering a bien « retrouvé » la structure sous-
58    jacente,
59  * indépendamment de l'ordre des centres.
60  */
61 def maxCenterDeviation(
62     trueCenters: Array[Array[Double]],
63     found:      Array[Array[Double]]
64 ): Double = {
65     var worst = 0.0
66     var i      = 0
67     while (i < trueCenters.length) {
68         val d = math.sqrt(Distance.nearest(trueCenters(i), found)._2)
69         if (d > worst) worst = d
70         i += 1
71     }
72     worst
73 }

```

Listing 10 – TestData.scala : générateur de données synthétiques à vérité terrain

```

1 package streamkm.core
2
3 import scala.util.Random
4
5 import org.scalatest.funsuite.AnyFunSuite
6 import org.scalatest.matchers.should.Matchers
7
8 import streamkm.TestData
9
10 class KMeansSpec extends AnyFunSuite with Matchers {
11
12     /** Convertit un PointsSoA en Array[Array[Double]] pour réutiliser TestData
13         tel quel
14      * (TestData reste en AoS, cf. décision prise pour CostEvaluatorSpec). */
15     private def toArrayOfArrays(store: PointsSoA): Array[Array[Double]] = {
16         val out = new Array[Array[Double]](store.n)
17         val tmp = new Array[Double](store.dimension)
18         var i = 0
19         while (i < store.n) {
20             store.copyCoordinatesInto(i, tmp)
21             out(i) = tmp.clone()
22             i += 1
23         }
24         out
25     }
26
27     // -----
28     // Test 1 - valeur analytique connue
29     // -----
30     test("cost : trois points colinéaires, k=1, valeur analytique") {

```

```

30     val pts    = PointsSoA.fromPoints(Array(Point.of(-1.0), Point.of(0.0), Point.
31         of(1.0)))
32     val zero    = PointsSoA.allocate(1, dimension = 1)
33     zero.append(Array(0.0), 1.0)
34
35     KMeans.cost(pts, zero) shouldBe 2.0 +- 1e-12
36
37     val model = KMeans.fit(pts, k = 1, nRestarts = 3, seed = 1L)
38     model.centers.coordinates(0, 0) shouldBe 0.0 +- 1e-9
39     model.cost shouldBe 2.0 +- 1e-9
40
41     pts.free(); zero.free(); model.centers.free()
42 }
43
44 // -----
45 // Test 2 - deux gaussiennes bien séparées
46 // -----
47 test("lloyd : retrouve deux gaussiennes séparées") {
48     val rng = new Random(7L)
49     val pts = Array.tabulate(2000) { i =>
50         val c = if (i % 2 == 0) 0.0 else 10.0
51         Point(Array(c + rng.nextGaussian() * 0.1, c + rng.nextGaussian() * 0.1))
52     }
53     val store = PointsSoA.fromPoints(pts)
54
55     val model = KMeans.fit(store, k = 2, nRestarts = 5, seed = 3L)
56     val dev    = TestData.maxCenterDeviation(Array(Array(0.0, 0.0), Array(10.0,
57         10.0)), toArrayOfArrays(model.centers))
58     dev should be < 0.05
59
60     store.free(); model.centers.free()
61 }
62
63 // -----
64 // Test 3 - monotonie du coût
65 // -----
66 test("lloyd : le coût ne remonte jamais") {
67     val mix      = TestData.gaussianMixture(n = 3000, k = 4, d = 3, seed = 99L)
68     val store     = PointsSoA.fromPoints(mix.points)
69     val badInit   = PointsSoA.allocate(4, dimension = 3)
70     var c = 0
71     while (c < 4) { badInit.append(Array(0.0, 0.0, 0.0), 1.0); c += 1 }
72
73     val before = KMeans.cost(store, badInit)
74     // lloyd consomme et libère 'badInit' (contrat établi lors de l'adaptation
75     // de KMeans) :
76     // on ne peut donc plus s'en servir après l'appel, d'où le calcul de 'before
77     // en premier.
78     val model = KMeans.lloyd(store, badInit, maxIter = 50)
79     model.cost should be <= before
80
81     store.free(); model.centers.free()
82 }
83
84 // -----
85 // Test 4 - pondération (LE test qui attrape le bug classique)
86 // -----
87 test("lloyd : un point de poids w équivaut à w copies du même point") {
88     val a = Array(0.0, 0.0)
89     val b = Array(10.0, 0.0)
90
91     val replicated = PointsSoA.fromPoints(Array(Point(a), Point(a), Point(a),
92         Point(b)))

```

```

88     val weighted    = PointsSoA.fromPoints(Array(Point(a, 3.0), Point(b, 1.0)))
89
90     def freshInit(): PointsSoA = {
91         val init = PointsSoA.allocate(1, dimension = 2)
92         init.append(Array(-5.0, 0.0), 1.0)
93         init
94     }
95
96     val mR = KMeans.lloyd(replicated, freshInit(), maxIter = 50)
97     val mW = KMeans.lloyd(weighted, freshInit(), maxIter = 50)
98
99     mR.centers.coordinates(0, 0) shouldBe 2.5 +- 1e-9
100    mW.centers.coordinates(0, 0) shouldBe 2.5 +- 1e-9
101    mW.cost shouldBe mR.cost +- 1e-9
102
103    replicated.free(); weighted.free()
104    mR.centers.free(); mW.centers.free()
105 }
106
107 // -----
108 // Test 5 - seeding
109 // -----
110 test("seedPlusPlus : produit k centres, bornés par n, de la bonne dimension")
111 {
112     val mix    = TestData.gaussianMixture(n = 500, k = 5, d = 4, seed = 21L)
113     val store = PointsSoA.fromPoints(mix.points)
114
115     val s = KMeans.seedPlusPlus(store, k = 5, new Random(1L))
116     s.n shouldBe 5
117     s.dimension shouldBe 4
118     s.free()
119
120     val tinyStore = PointsSoA.fromPoints(Array(Point.of(0.0), Point.of(1.0)))
121     val s2 = KMeans.seedPlusPlus(tinyStore, k = 10, new Random(1L))
122     s2.n shouldBe 2
123     s2.free()
124
125     store.free(); tinyStore.free()
126 }
127
128 test("seedPlusPlus : le seeding D2 sépare les groupes éloignés") {
129     val rng = new Random(5L)
130     val pts = Array.tabulate(900) { i =>
131         val c = (i % 3) * 1000.0
132         Point(Array(c + rng.nextGaussian()))
133     }
134     val store = PointsSoA.fromPoints(pts)
135
136     var successes = 0
137     (0 until 20).foreach { t =>
138         val s = KMeans.seedPlusPlus(store, 3, new Random(t.toLong))
139         val groups = (0 until s.n).map(i => math.round(s.coordinates(i, 0) /
140             1000.0)).distinct
141         if (groups.length == 3) successes += 1
142         s.free()
143     }
144     successes should be >= 18
145
146     store.free()
147 }
148
149 // -----
150 // Test 6 - fit garde bien le meilleur redémarrage

```

```

149 // -----
150 test("fit : le coût décroît (au sens large) avec le nombre de redémarrages") {
151     val mix    = TestData.gaussianMixture(n = 2000, k = 6, d = 2, sigma = 2.0,
152         seed = 33L)
153     val store = PointsSoA.fromPoints(mix.points)
154
155     val one    = KMeans.fit(store, k = 6, nRestarts = 1, seed = 8L)
156     val five   = KMeans.fit(store, k = 6, nRestarts = 5, seed = 8L)
157     five.cost should be <= one.cost + 1e-9
158
159     store.free(); one.centers.free(); five.centers.free()
160 }

```

Listing 11 – core/KMeansSpec.scala

```

1 package streamkm.coreset
2
3 import scala.util.Random
4
5 import org.scalatest.funsuite.AnyFunSuite
6 import org.scalatest.matchers.should.Matchers
7
8 import streamkm.TestData
9 import streamkm.core.{KMeans, Point, PointsSoA}
10
11 class CoresetSpec extends AnyFunSuite with Matchers {
12
13     private def totalMass(store: PointsSoA): Double = {
14         var sum = 0.0
15         var i = 0
16         while (i < store.n) { sum += store.weight(i); i += 1 }
17         sum
18     }
19
20     // =====
21     // CoresetTree
22     // =====
23
24     test("CoresetTree : renvoie l'ensemble tel quel si n <= m") {
25         val store = PointsSoA.fromPoints(TestData.gaussianMixture(n = 50, k = 3, d =
26             2, seed = 1L).points)
27         val cs    = CoresetTree.build(store, coresetSize = 100, new Random(1L))
28         cs.n shouldBe 50
29         totalMass(cs) shouldBe 50.0 +- 1e-9
30         cs.free() // == store ici (renvoyé tel quel, n <= m) : un seul free() suffit
31     }
32
33     test("CoresetTree : produit au plus m points") {
34         for (m <- Seq(10, 100, 1000)) {
35             val store = PointsSoA.fromPoints(TestData.gaussianMixture(n = 5000, k = 5,
36                 d = 3, seed = 2L).points)
37             val cs    = CoresetTree.build(store, m, new Random(m.toLong))
38             cs.n should be <= m
39             cs.free()
40         }
41     }
42
43     test("CoresetTree : conserve exactement la masse totale") {
44         for (m <- Seq(10, 100, 500)) {
45             val store = PointsSoA.fromPoints(TestData.gaussianMixture(n = 5000, k = 5,
46                 d = 3, seed = 3L).points)
47             val cs    = CoresetTree.build(store, m, new Random(m.toLong))
48             withClue(s"m=$m : ") { totalMass(cs) shouldBe 5000.0 +- 1e-6 }
49         }
50     }
51 }

```

```

46     cs.free()
47 }
48 }
49
50 test("CoresetTree : conserve la masse sur des points déjà pondérés") {
51     val rng = new Random(4L)
52     val pts = Array.tabulate(3000)(_ =>
53         Point(Array(rng.nextGaussian() * 10, rng.nextGaussian() * 10), 1.0 + rng.
54             nextInt(9))
55     )
56     val store = PointsSoA.fromPoints(pts)
57     val expected = totalMass(store)
58     // 'store' sera consommé par build ; il faut donc calculer 'expected' avant
59     // l'appel.
60     val cs = CoresetTree.build(store, coresetSize = 200, new Random(5L))
61     totalMass(cs) shouldBe expected +- 1e-6
62     cs.free()
63 }
64
65 test("CoresetTree : gère un ensemble de points tous identiques") {
66     val store = PointsSoA.fromPoints(Array.fill(1000)(Point(Array(3.0, 4.0))))
67     val cs = CoresetTree.build(store, coresetSize = 50, new Random(6L))
68     cs.n should be <= 50
69     totalMass(cs) shouldBe 1000.0 +- 1e-9
70     var i = 0
71     while (i < cs.n) {
72         cs.coordinates(i, 0) shouldBe 3.0
73         cs.coordinates(i, 1) shouldBe 4.0
74         i += 1
75     }
76     cs.free()
77 }
78
79 test("CoresetTree : le coreset préserve la qualité du clustering") {
80     val mix = TestData.gaussianMixture(n = 20000, k = 5, d = 2, sigma =
81         0.5, seed = 7L)
82     val fullStore = PointsSoA.fromPoints(mix.points)
83     val full = KMeans.fit(fullStore, k = 5, nRestarts = 5, seed = 11L)
84
85     val coresetSrc = PointsSoA.fromPoints(mix.points) // second store : le
86     // premier a servi à 'full', indépendant du coreset
87     val cs = CoresetTree.build(coresetSrc, coresetSize = 200, new Random
88         (12L))
89     val onCoreset = KMeans.fit(cs, k = 5, nRestarts = 5, seed = 11L)
90     val costViaCores = KMeans.cost(fullStore, onCoreset.centers)
91
92     withClue(s"complet=${full.cost}, via coreset=$costViaCores : ") {
93         costViaCores should be <= full.cost * 1.10
94     }
95
96     fullStore.free(); full.centers.free(); onCoreset.centers.free()
97 }
98
99 // =====
100 // BucketManager
101 // =====
102
103 test("BucketManager : compte correctement les points consommés") {
104     val bm = BucketManager(coresetSize = 100, dimension = 2, seed = 1L)
105     val store = PointsSoA.fromPoints(TestData.gaussianMixture(n = 5000, k = 4, d
106         = 2, seed = 8L).points)
107     bm.insertAll(store)
108     bm.numPointsSeen shouldBe 5000.0 +- 1e-9

```



```

103     store.free(); bm.free()
104 }
105
106 test("BucketManager : respecte les invariants de taille des buckets") {
107     val bm = BucketManager(coresetSize = 64, dimension = 2, seed = 2L)
108     val pts = TestData.gaussianMixture(n = 4000, k = 4, d = 2, seed = 9L).points
109     pts.foreach { p =>
110         bm.insert(p.coords, p.weight)
111         bm.checkInvariants() shouldBe true
112     }
113     bm.free()
114 }
115
116 test("BucketManager : le nombre de niveaux croît logarithmiquement") {
117     val m = 100
118     val bm = BucketManager(m, dimension = 2, seed = 3L)
119     val store = PointsSoA.fromPoints(TestData.gaussianMixture(n = 100000, k = 3,
120         d = 2, seed = 10L).points)
121     bm.insertAll(store)
122     bm.numLevels should be <= 16
123     store.free(); bm.free()
124 }
125
126 test("BucketManager : union() et extractCoreset() conservent la masse") {
127     val bm = BucketManager(coresetSize = 128, dimension = 3, seed = 4L)
128     val store = PointsSoA.fromPoints(TestData.gaussianMixture(n = 10000, k = 5,
129         d = 3, seed = 11L).points)
130     bm.insertAll(store)
131
132     val u = bm.union()
133     totalMass(u) shouldBe 10000.0 +- 1e-6
134     u.free()
135
136     val ec = bm.extractCoreset()
137     totalMass(ec) shouldBe 10000.0 +- 1e-6
138     ec.n should be <= 128
139     ec.free()
140
141     store.free(); bm.free()
142 }
143
144 test("BucketManager : extractCoreset ne détruit pas l'état") {
145     val bm = BucketManager(coresetSize = 64, dimension = 2, seed = 5L)
146     val store1 = PointsSoA.fromPoints(TestData.gaussianMixture(n = 5000, k = 3,
147         d = 2, seed = 12L).points)
148     bm.insertAll(store1)
149
150     val first = bm.extractCoreset()
151     first.n should be <= 64
152     first.free()
153
154     val store2 = PointsSoA.fromPoints(TestData.gaussianMixture(n = 5000, k = 3,
155         d = 2, seed = 13L).points)
156     bm.insertAll(store2)
157     val second = bm.extractCoreset()
158
159     bm.numPointsSeen shouldBe 10000.0 +- 1e-9
160     totalMass(second) shouldBe 10000.0 +- 1e-6
161
162     second.free(); store1.free(); store2.free(); bm.free()
163 }
164
165 test("BucketManager : merge conserve la masse des deux opérands") {

```

```

162     val a = BucketManager(coresetSize = 100, dimension = 2, seed = 6L)
163     val b = BucketManager(coresetSize = 100, dimension = 2, seed = 7L)
164     val storeA = PointsSoA.fromPoints(TestData.gaussianMixture(n = 4000, k = 4,
165         d = 2, seed = 14L).points)
166     val storeB = PointsSoA.fromPoints(TestData.gaussianMixture(n = 6000, k = 4,
167         d = 2, seed = 15L).points)
168     a.insertAll(storeA); b.insertAll(storeB)
169
170     val merged = a.merge(b)
171     merged.numPointsSeen shouldBe 10000.0 +- 1e-9
172     val mu = merged.union()
173     totalMass(mu) shouldBe 10000.0 +- 1e-6
174     mu.free()
175
176     a.numPointsSeen shouldBe 4000.0 +- 1e-9
177     b.numPointsSeen shouldBe 6000.0 +- 1e-9
178
179     storeA.free(); storeB.free(); a.free(); b.free(); merged.free()
180 }
181
182 test("BucketManager : insertion refusée après merge") {
183     val a = BucketManager(coresetSize = 32, dimension = 2, seed = 8L)
184     val b = BucketManager(coresetSize = 32, dimension = 2, seed = 9L)
185     val storeA = PointsSoA.fromPoints(TestData.gaussianMixture(n = 500, k = 2, d
186         = 2, seed = 16L).points)
187     val storeB = PointsSoA.fromPoints(TestData.gaussianMixture(n = 500, k = 2, d
188         = 2, seed = 17L).points)
189     a.insertAll(storeA); b.insertAll(storeB)
190
191     an[IllegalStateException] should be thrownBy a.merge(b).insert(Array(0.0,
192         0.0), 1.0)
193
194     storeA.free(); storeB.free(); a.free(); b.free()
195 }
196
197 // =====
198 // Bout en bout : StreamKM++ séquentiel
199 // =====
200
201 test("StreamKM++ séquentiel : qualité comparable au k-means++ batch") {
202     val k = 5
203     val m = BucketManager.recommendedM(k)
204     val mix = TestData.gaussianMixture(n = 50000, k = k, d = 2, sigma = 0.5,
205         seed = 18L)
206
207     val bm = BucketManager(m, dimension = 2, seed = 19L)
208     val store = PointsSoA.fromPoints(mix.points)
209     bm.insertAll(store)
210
211     val finalCoreset = bm.union()
212     val streamModel = KMeans.fit(finalCoreset, k, nRestarts = 5, seed = 20L)
213     val streamCost = KMeans.cost(store, streamModel.centers)
214
215     val refStore = PointsSoA.fromPoints(mix.points)
216     val batchModel = KMeans.fit(refStore, k, nRestarts = 5, seed = 20L)
217     val batchCost = batchModel.cost
218
219     withClue(s"batch=$batchCost, stream=$streamCost : ") {
220         streamCost should be <= batchCost * 1.10
221     }
222
223     val centersArr = {
224         val out = new Array[Array[Double]](streamModel.centers.n)

```

```

219     val tmp = new Array[Double](streamModel.centers.dimension)
220     var i = 0
221     while (i < streamModel.centers.n) { streamModel.centers.
        copyCoordinatesInto(i, tmp); out(i) = tmp.clone(); i += 1 }
222     out
223 }
224 TestData.maxCenterDeviation(mix.trueCenters, centersArr) should be < 0.5
225
226 store.free(); finalCoreset.free(); streamModel.centers.free()
227 refStore.free(); batchModel.centers.free(); bm.free()
228 }
229 }

```

Listing 12 – coreset/CoresetSpec.scala

```

1 package streamkm.spark
2
3 import org.apache.spark.sql.{SQLContext, SparkSession}
4 import org.apache.spark.sql.execution.streaming.MemoryStream
5 import org.scalatest.BeforeAndAfterAll
6 import org.scalatest.funsuite.AnyFunSuite
7 import org.scalatest.matchers.should.Matchers
8
9 import streamkm.TestData
10 import streamkm.core.{KMeans, KMeansModel, PointsSoA}
11 import streamkm.coreset.BucketManager
12
13 private[spark] object CoresetErrorModel {
14     // Inchangé - aucune dépendance à Point/PointsSoA, purement arithmétique.
15     def scaleFactor(numPartitions: Int, depth: Int): Int =
16         math.max(2, math.ceil(math.pow(numPartitions.toDouble, 1.0 / depth)).toInt)
17
18     def worstCaseCompositionLevels(numPartitions: Int, depth: Int): Int = {
19         val scale = scaleFactor(numPartitions, depth)
20         depth * (scale - 1)
21     }
22
23     def compoundedTolerance(epsPerLevel: Double, levels: Int): Double =
24         math.pow(1.0 + epsPerLevel, levels) - 1.0
25
26     def treeReduceTolerance(numPartitions: Int, depth: Int, epsPerLevel: Double =
27         0.10): Double =
28         compoundedTolerance(epsPerLevel, worstCaseCompositionLevels(numPartitions,
29             depth))
30 }
31
32 class StreamKMSpec extends AnyFunSuite with Matchers with BeforeAndAfterAll {
33
34     @transient private var spark: SparkSession = _
35
36     override def beforeAll(): Unit = {
37         spark = SparkSession.builder()
38             .appName("StreamKMSpec")
39             .master("local[4]")
40             .config("spark.ui.enabled", "false")
41             .config("spark.sql.shuffle.partitions", "4")
42             .getOrCreate()
43     }
44
45     override def afterAll(): Unit = {
46         if (spark != null) spark.stop()
47     }
48 }

```

```

47  /** Reconstitue un 'PointsSoA', exécute 'f', libère, renvoie le résultat de 'f'
48  * Évite de répéter le triplet fromPoints/appele/free à chaque test. */
49  private def withStore[A](points: Array[streamkm.core.Point])(f: PointsSoA => A
50    ): A = {
51    val store = PointsSoA.fromPoints(points)
52    try f(store) finally store.free()
53  }
54
55  // =====
56  // partialCoresets
57  // =====
58  test("partialCoresets : un coreset par partition, masse totale conservée") {
59    val mix = TestData.gaussianMixture(n = 8000, k = 4, d = 2, seed = 31L)
60    val rdd = spark.sparkContext.parallelize(mix.points, numSlices = 4)
61
62    val params = StreamKMParams(k = 4, m = 200, dim = 2)
63    // partialCoresets renvoie désormais RDD[(Array[Point], Long)] - le Long est
64    // la
65    // graine de partition, cf. décision "indépendance des graines" (
66    // StreamKMPlusPlus).
67    val partials = StreamKMPlusPlus.partialCoresets(rdd, params).collect()
68
69    partials.length shouldBe 4
70    partials.foreach { case (points, _) => points.length should be <= 200 }
71
72    val totalMass = partials.iterator.flatMap(_._1).map(_._2.weight).sum
73    totalMass shouldBe 8000.0 +- 1e-6
74  }
75
76  // =====
77  // mergeCoresets
78  // =====
79  test("mergeCoresets : conserve la masse totale, quel que soit le nombre de
80    partitions") {
81    val mix = TestData.gaussianMixture(n = 10000, k = 5, d = 2, seed = 32L)
82    val params = StreamKMParams(k = 5, m = BucketManager.recommendedM(5), dim =
83      2)
84
85    for (nParts <- Seq(1, 2, 8)) {
86      val rdd = spark.sparkContext.parallelize(mix.points, numSlices =
87        nParts)
88      val coreset = StreamKMPlusPlus.mergeCoresets(rdd, params) // Array[Point],
89      // sérialisable
90      withClue(s"nParts=$nParts : ") {
91        coreset.map(_._2.weight).sum shouldBe 10000.0 +- 1e-6
92        coreset.length should be <= params.m
93      }
94    }
95  }
96
97  // =====
98  // LE TEST LE PLUS IMPORTANT : non-régression distribué vs séquentiel
99  // =====
100  test("mergeCoresets sur 8 partitions produit un clustering de qualité
    comparable au séquentiel") {
    val k = 5
    val m = BucketManager.recommendedM(k)
    val dim = 2
    val mix = TestData.gaussianMixture(n = 40000, k = k, d = dim, sigma =

```

```

    0.5, seed = 33L)
101 val numParts = 8
102 val treeDepth = 2
103
104 // Référence séquentielle : un seul BucketManager, tout le flux d'un coup.
105 val (seqCost, seqCenters) = withStore(mix.points) { seqStore =>
106     val seqBm = BucketManager(m, dim, seed = 40L)
107     seqBm.insertAll(seqStore)
108     val seqCoreset = seqBm.extractCoreset()
109     val seqModel = KMeans.fit(seqCoreset, k, nRestarts = 5, seed = 41L)
110     seqCoreset.free()
111     val cost = KMeans.cost(seqStore, seqModel.centers)
112     seqBm.free()
113     (cost, seqModel.centers)
114 }
115
116 // Version distribuée : mêmes données, réparties sur 'numParts' partitions
117 // Spark.
118 val rdd = spark.sparkContext.parallelize(mix.points, numSlices =
119     numParts)
120 val distParams = StreamKMParams(k = k, m = m, dim = dim, seed = 40L)
121 val distCoresetPts = StreamKMPlusPlus.mergeCoresets(rdd, distParams, depth =
122     treeDepth)
123 val distCoreset = PointsSoA.fromPoints(distCoresetPts)
124 val distModel = KMeans.fit(distCoreset, k, nRestarts = 5, seed = 41L)
125 distCoreset.free()
126
127 val distCost = withStore(mix.points)(store => KMeans.cost(store, distModel.
128     centers))
129
130 val tolerance = CoresetErrorModel.treeReduceTolerance(numParts, treeDepth)
131
132 withClue(
133     s"séquentiel=$seqCost, distribué=$distCost, tolérance théorique=${
134         tolerance * 100}%% : "
135 ) {
136     distCost should be <= seqCost * (1.0 + tolerance)
137 }
138
139 val distCentersArr = toArrayOfArrays(distModel.centers)
140 TestData.maxCenterDeviation(mix.trueCenters, distCentersArr) should be < 2.0
141
142 seqCenters.free(); distModel.centers.free()
143 }
144
145 test("mergeCoresets : le coût ne se dégrade pas franchement entre 1 et 8
146     partitions") {
147     val k = 4
148     val m = BucketManager.recommendedM(k)
149     val dim = 2
150     val mix = TestData.gaussianMixture(n = 20000, k = k, d = dim, sigma = 0.5,
151         seed = 34L)
152
153     val params = StreamKMParams(k = k, m = m, dim = dim, seed = 50L)
154
155     def costWith(nParts: Int): Double = {
156         val rdd = spark.sparkContext.parallelize(mix.points, numSlices =
157             nParts)
158         val coresetPts = StreamKMPlusPlus.mergeCoresets(rdd, params)
159         val coresetStore = PointsSoA.fromPoints(coresetPts)
160         val model = KMeans.fit(coresetStore, k, nRestarts = 5, seed = 51L)
161         coresetStore.free()
162         val result = withStore(mix.points)(store => KMeans.cost(store, model.

```

```

        centers))
155     model.centers.free()
156     result
157 }
158
159 val cost1 = costWith(1)
160 val cost8 = costWith(8)
161
162 withClue(s"1 partition=$cost1, 8 partitions=$cost8 : ") {
163     math.abs(cost1 - cost8) should be <= 0.3 * math.max(cost1, cost8)
164 }
165 }
166
167 test("partialCoresets et mergeCoresets utilisent des graines distinctes par
    partition") {
168     val pts      = TestData.gaussianMixture(n = 2000, k = 3, d = 2, seed = 60L).
        points
169     val rdd      = spark.sparkContext.parallelize(pts ++ pts, numSlices = 2)
170     val params   = StreamKMParams(k = 3, m = 50, dim = 2)
171
172     val partials = StreamKMPlusPlus.partialCoresets(rdd, params).collect()
173     partials.length shouldBe 2
174     // Déstructuration : on ne compare que les points, la graine de partition (
        second
175     // élément du tuple) n'a pas à être identique - au contraire, c'est
        justement ce
176     // qu'on vérifie indirectement via la divergence des coreset produits.
177     val coordsA = partials(0)._1.map(_.coords.toSeq).toSet
178     val coordsB = partials(1)._1.map(_.coords.toSeq).toSet
179     coordsA should not equal coordsB
180 }
181
182 // =====
183 // ÉTAT STREAMING DANS LE TEMPS
184 // =====
185
186 test(
187     "état streaming : l'insertion multi micro-batches conserve exactement la
        masse, " +
188     "sans double-comptage ni perte"
189 ) {
190     val k      = 4
191     val m      = BucketManager.recommendedM(k)
192     val dim    = 2
193     val mix    = TestData.gaussianMixture(n = 30000, k = k, d = dim, sigma = 0.5,
        seed = 70L)
194
195     val chunks = mix.points.grouped(10000).toArray
196     chunks.length shouldBe 3
197
198     val global = BucketManager(m, dim, seed = 71L)
199     var cumulativeExpected = 0.0
200
201     chunks.zipWithIndex.foreach { case (chunk, batchId) =>
202         val rdd      = spark.sparkContext.parallelize(chunk, numSlices = 4)
203         val partial  = StreamKMPlusPlus.mergeCoresets(rdd, StreamKMParams(k = k, m
            = m, dim = dim, seed = 71L + batchId))
204         // partial est Array[Point] (AoS, sérialisable) - insertion point par
            point dans
205         // le BucketManager global, qui attend (coords, weight).
206         var i = 0
207         while (i < partial.length) { global.insert(partial(i).coords, partial(i).
            weight); i += 1 }

```

```

208         cumulativeExpected += chunk.length.toDouble
209         withClue(s"après le micro-batch $batchId : ") {
210             global.numPointsSeen shouldBe cumulativeExpected +- 1e-6
211         }
212     }
213 }
214
215 val finalCoreset = global.extractCoreset()
216 var mass = 0.0
217 var i = 0
218 while (i < finalCoreset.n) { mass += finalCoreset.weight(i); i += 1 }
219 mass shouldBe 30000.0 +- 1e-6
220
221 finalCoreset.free(); global.free()
222 }
223
224 // Ignoré sous Windows sans winutils.exe/HADOOP_HOME - cf. CLAUDE.md, décision
225 13.
226 test("run() sur plusieurs micro-batches produit une qualité comparable à un
227      traitement en un seul bloc") {
228     val k = 4
229     val m = BucketManager.recommendedM(k)
230     val dim = 2
231     val mix = TestData.gaussianMixture(n = 30000, k = k, d = dim, sigma = 0.5,
232                                       seed = 72L)
233     val chunks = mix.points.grouped(10000).toArray
234     chunks.length shouldBe 3
235
236     val sparkSession: SparkSession = spark
237     implicit val sqlCtx: SQLContext = sparkSession.sqlContext
238     import sparkSession.implicits._
239     val source = MemoryStream(Array[Double])
240     val df = source.toDF().withColumnRenamed("value", "features")
241
242     val models = scala.collection.mutable.ArrayBuffer.empty[KMeansModel]
243     val query = StreamKMPlusPlus.run(df, StreamKMParams(k = k, m = m, dim = dim,
244                                                         , seed = 73L), models += _)
245
246     try {
247         chunks.foreach { chunk =>
248             source.addData(chunk.map(_.coords))
249             query.processAllAvailable()
250         }
251     } finally {
252         query.stop()
253     }
254
255     models should have length 3
256
257     val streamCost = withStore(mix.points)(store => KMeans.cost(store, models.
258                                                                last.centers))
259     val lastCentersArr = toArrayOfArrays(models.last.centers)
260     TestData.maxCenterDeviation(mix.trueCenters, lastCentersArr) should be < 2.0
261
262     // Référence : les mêmes points, en un seul bloc.
263     val refBm = BucketManager(m, dim, seed = 71L)
264     val rdd = spark.sparkContext.parallelize(mix.points, numSlices = 4)
265     val refPartial = StreamKMPlusPlus.mergeCoresets(rdd, StreamKMParams(k = k, m
266                               = m, dim = dim, seed = 71L))
267     var i = 0
268     while (i < refPartial.length) { refBm.insert(refPartial(i).coords,
269                                                  refPartial(i).weight); i += 1 }
270     val refCoreset = refBm.extractCoreset()

```

```

264     val refModel    = KMeans.fit(refCoreset, k, nRestarts = 5, seed = 73L)
265     refCoreset.free()
266     val refCost = withStore(mix.points)(store => KMeans.cost(store, refModel.
        centers))
267
268     val tolerance = CoresetErrorModel.treeReduceTolerance(numPartitions = 4,
        depth = 2)
269     withClue(s"streaming (3 micro-batches)=$streamCost, référence (1 bloc)=
        $refCost : ") {
270         streamCost should be <= refCost * (1.0 + tolerance)
271     }
272
273     models.foreach(_ .centers.free())
274     refModel.centers.free(); refBm.free()
275 }
276
277 /** Conversion PointsSoA → Array[Array[Double]], pour réutiliser TestData.
        maxCenterDeviation
278 * tel quel (resté en AoS). */
279 private def toArrayOfArrays(store: PointsSoA): Array[Array[Double]] = {
280     val out = new Array[Array[Double]](store.n)
281     val tmp = new Array[Double](store.dimension)
282     var i = 0
283     while (i < store.n) { store.copyCoordinatesInto(i, tmp); out(i) = tmp.clone
        (); i += 1 }
284     out
285 }
286 }

```

Listing 13 – spark/StreamKMSpec.scala

```

1 package streamkm.spark
2
3 import org.apache.spark.sql.SparkSession
4 import org.scalatest.BeforeAndAfterAll
5 import org.scalatest.funSuite.AnyFunSuite
6 import org.scalatest.matchers.should.Matchers
7
8 import streamkm.TestData
9 import streamkm.core.{KMeans, PointsSoA}
10
11 class CostEvaluatorSpec extends AnyFunSuite with Matchers with BeforeAndAfterAll
    {
12
13     @transient private var spark: SparkSession = _
14
15     override def beforeAll(): Unit = {
16         spark = SparkSession.builder()
17             .appName("CostEvaluatorSpec")
18             .master("local[4]")
19             .config("spark.ui.enabled", "false")
20             .getOrCreate()
21     }
22
23     override def afterAll(): Unit = {
24         if (spark != null) spark.stop()
25     }
26
27     test("cost : égal (à la précision flottante près) au calcul séquentiel KMeans.
        cost") {
28         val mix          = TestData.gaussianMixture(n = 12000, k = 4, d = 3, seed =
            80L)
29         val pointsStore = PointsSoA.fromPoints(mix.points)
30         val model       = KMeans.fit(pointsStore, k = 4, nRestarts = 3, seed = 81L)

```



```

31     val refCost      = KMeans.cost(pointsStore, model.centers)
32
33     val rdd          = spark.sparkContext.parallelize(mix.points, numSlices = 6)
34     val distCost     = CostEvaluator.cost(rdd, model.centers)
35
36     distCost shouldBe refCost +- (refCost.abs * 1e-9 + 1e-9)
37
38     pointsStore.free()
39     model.centers.free()
40 }
41
42 test("cost : invariant au nombre de partitions") {
43     val mix          = TestData.gaussianMixture(n = 9000, k = 3, d = 2, seed = 82
44         L)
45     val pointsStore = PointsSoA.fromPoints(mix.points)
46     val model       = KMeans.fit(pointsStore, k = 3, nRestarts = 3, seed = 83L)
47
48     val costs = Seq(1, 3, 9).map { nParts =>
49         val rdd = spark.sparkContext.parallelize(mix.points, numSlices = nParts)
50         nParts -> CostEvaluator.cost(rdd, model.centers)
51     }
52
53     val values = costs.map(_._2)
54     withClue(s"coûts par partitionnement : $costs : ") {
55         values.max - values.min should be <= (values.max.abs * 1e-9 + 1e-9)
56     }
57
58     pointsStore.free()
59     model.centers.free()
60 }
61
62 test("cost : détecte une régression grossière (mauvais centres = coût très sup
63     érieur)") {
64     val mix          = TestData.gaussianMixture(n = 6000, k = 4, d = 2, sigma =
65         0.5, seed = 84L)
66     val pointsStore = PointsSoA.fromPoints(mix.points)
67     val goodModel   = KMeans.fit(pointsStore, k = 4, nRestarts = 5, seed = 85L
68         )
69
70     // Centres délibérément mauvais - construits directement en PointsSoA plutôt
71     // que
72     // via Array[Array[Double]] (ancienne API), pour rester cohérent avec la
73     // signature
74     // que CostEvaluator.cost attend désormais.
75     val badCenters = PointsSoA.allocate(4, dimension = 2)
76     var i = 0
77     while (i < 4) { badCenters.append(Array(1000.0, 1000.0), 1.0); i += 1 }
78
79     val rdd          = spark.sparkContext.parallelize(mix.points, numSlices = 4)
80     val goodCost     = CostEvaluator.cost(rdd, goodModel.centers)
81     val badCost      = CostEvaluator.cost(rdd, badCenters)
82
83     badCost should be > goodCost * 10.0
84
85     pointsStore.free()
86     goodModel.centers.free()
87     badCenters.free()
88 }
89
90 test("cost : accepte directement un KMeansModel (surcharge de confort)") {
91     val mix          = TestData.gaussianMixture(n = 3000, k = 2, d = 2, seed = 86
92         L)
93     val pointsStore = PointsSoA.fromPoints(mix.points)

```

```
87     val model          = KMeans.fit(pointsStore, k = 2, nRestarts = 3, seed = 87L)
88     val rdd             = spark.sparkContext.parallelize(mix.points, numSlices = 3)
89
90     CostEvaluator.cost(rdd, model) shouldBe CostEvaluator.cost(rdd, model.
      centers)
91
92     pointsStore.free()
93     model.centers.free()
94   }
95 }
```

Listing 14 – spark/CostEvaluatorSpec.scala

Bibliographie

- [1] Theodoros Bitsakis. *Clustering Big Data Streams in Apache Flink*. Diploma thesis, 2018.
- [2] Marcel R. Ackermann, Marcus Mörtens, Christiane Raupach, Kamil Swierkot, Christiane Lammersen, Christian Sohler. *StreamKM++ : A clustering algorithm for data streams*. ACM Journal of Experimental Algorithmics (JEA), 17, 2012.
- [3] David Arthur, Sergei Vassilvitskii. *k-means++ : The Advantages of Careful Seeding*. Proceedings of the 18th annual ACM-SIAM symposium on Discrete algorithms, 2007.